

---

# Can Power Draw Constrain Covert Compute?

---

Tom Kimpson<sup>1,2</sup> Mauricio Baker<sup>3</sup> Emlyn Graham<sup>2</sup> Anjay Friedman<sup>3</sup> Maximus Rafla<sup>3</sup> Coby Joseph<sup>3</sup>

## Abstract

International agreements on frontier AI will need verification: an outside party must be able to check how much computation an operator actually ran. The chip’s own telemetry reads silicon the operator controls, so analogue sensors placed off the chip have been proposed instead, e.g. a meter on the power feed, or an electromagnetic antenna or a thermal camera beside the rack. How tightly such sensors constrain compute against an operator who actively tries to hide computation is unknown. We derive a closed form for  $\beta$ , the largest hidden computation a power trace cannot exclude, in units of declared machine capacity.  $\beta$  is set by how widely the energy cost of an operation can vary, how cheaply a hidden operation can run, and how much idle draw the meter must allow for; we measure each on NVIDIA A100 GPUs. A verifier that reads the power and checks the declared numeric format gets  $\beta = 1.16$ : more than a full machine of hidden compute. That bound is loose. We hid 0.19 to 0.41 machines of undeclared work undetected. Re-running the declared work, restricting hidden work to useful computation, and pinning the idle draw together lower  $\beta$  to 0.059. A power meter alone is therefore a weak bound on compute, and the closed form prices what each further capability buys. It gives a quantitative basis for what off-chip sensing can contribute to verifying agreements on AI compute.

## 1. Introduction

As frontier AI systems grow more capable (Sevilla et al., 2022; Bengio et al., 2026), governance proposals increasingly rely on *verification*: the ability of an outside party to check what a monitored operator is doing (Reuel et al., 2025;

Baker et al., 2025). One proposed mechanism is a threshold on compute (Sastry et al., 2024). Above a specified number of floating-point operations (FLOP), a training run triggers registration, reporting, or audit obligations (Shavit, 2023; Baker et al., 2025; Heim & Koessler, 2024). For example, U.S. Executive Order 14110 temporarily required reports on models trained using more than  $10^{26}$  integer or floating-point operations (The White House, 2023; 2025).

Compute verification policies need to be robust to adversarial operators who do not behave honestly, but instead try to subvert the threshold by hiding computation. Verification methods that rely on passive telemetry such as utilisation counters, on-chip clocks or memory traffic read silicon the operator controls. Consequently they rely on tamper-resistant read paths and attested monitoring firmware that do not yet exist (O’Gara et al., 2025; Ansari, 2026; Rahman & Tajdari, 2026). Cryptographic proof-of-training protocols (Jia et al., 2021; Garg et al., 2023) depend on the operator’s cooperation. Active challenge–response schemes (Monfared et al., 2026) run their probes on the operator’s own untrusted hardware.

“Off-chip” analogue sensors have been suggested as an alternative verification method (Baker et al., 2025). This involves measurement hardware physically separate from the monitored chips such as a meter on the power feed, an antenna beside the rack, a microphone, or a thermal camera. Each reads a physical side channel such as power, electromagnetic, acoustic, or thermal emissions and so its measurements reach the verifier without passing through any silicon or software the operator controls. Such sensors could be retrofitted to existing data centres and physically secured against tampering by the adversary.

There are reasons for optimism: in non-adversarial settings, physical emissions are known to leak workload information. Emissions let an observer recover model architectures and hyperparameters (Hu et al., 2020; Maia et al., 2022; Gao et al., 2023; Chaudhuri et al., 2025), and software-visible counters and timing let a co-resident process do the same (Naghbijouybari et al., 2018; Wei et al., 2020). On-node telemetry and on-chip counters classify workloads (Tang et al., 2022; Xu et al., 2026; Gangwal et al., 2019), and power draw betrays anomalous computation (Clark et al., 2013) and running GPU activity (Arefin & Serwadda, 2024);

---

<sup>1</sup>School of Mathematics and Statistics, The University of Melbourne, Parkville, VIC, Australia <sup>2</sup>Machine Alignment, Transparency & Security (MATS) <sup>3</sup>RAND Corporation. Correspondence to: Tom Kimpson <tom.kimpson@unimelb.edu.au>.

a related body of work characterises power draw for GPUs and data centres (Patel et al., 2023; Choukse et al., 2025; Latif et al., 2025). However these are forward models, fitted largely in cooperative conditions, i.e. they map known workloads to their emissions. Verification poses the inverse problem (i.e. emissions to workload) against an operator actively trying to evade the sensor. It is an open question how well analogue-sensor measurements can constrain compute in that setting. This paper addresses that question for one channel: power, the channel most directly linked to computation, since every operation costs energy and total energy therefore budgets total work. We defer the study of other channels to a future work.

This paper is organised as follows. Section 2 sets up the problem mathematically. Section 3 derives a floor on the covert compute a power trace admits, in closed form, from a handful of physical bands. Section 4 empirically determines how large that floor is by measuring the bands directly on NVIDIA A100 hardware. Section 5 shows how additional information gained by the verifier beyond the power trace (e.g a precision declaration) pushes the floor lower. We close with a discussion in Section 6.

## 2. Setup and threat model

### 2.1. The forward model

Fix an observation window  $t \in [0, T]$  and let  $F(t)$  be the instantaneous rate of arithmetic operations, bounded by the hardware capacity  $F_{\max}$ . The forward model of  $F(t)$  to power follows from the CMOS power law (Chandrakasan et al., 1992); chip power splits into a switching term and a static term,

$$P = \underbrace{\alpha C V^2 f}_{\text{dynamic}} + P_{\text{stat}}, \quad (1)$$

with  $V$  the supply voltage,  $f$  the clock frequency,  $C$  the total capacitance of the logic, and  $\alpha$  the activity factor: the fraction of  $C$  that actually toggles in a given cycle ( $0 \leq \alpha \leq 1$ ), so that  $\alpha C$  is the capacitance switched per clock tick. A datapath that retires  $N_{\text{op}}$  nominal operations per cycle runs at operation rate  $F = N_{\text{op}} f$ . Substituting  $f = F/N_{\text{op}}$  and collecting constants gives the forward model

$$P(t) = r(t) F(t) + P_0(t), \quad r \equiv \alpha \left( \frac{C}{N_{\text{op}}} \right) V^2, \quad (2)$$

where  $P_0$  collects  $P_{\text{stat}}$  and the remaining non-compute overhead (idle draw, cooling, I/O). We call  $r$  the *exchange rate* [J/op]. The clock frequency has dropped out: what an operation costs does not depend on how fast the operations run, only on what each switching event costs and how many operations it delivers. Appendix A gives this derivation in full.

The exchange rate  $r(t)$  is not a constant of the device but

a property of the execution, and it changes whenever the execution does. Take its three factors in turn. The activity factor  $\alpha$  depends on the operand values being multiplied (Chandrakasan et al., 1992). The capacitance switched per operation,  $C/N_{\text{op}}$ , depends on the numeric format (which sets both the datapath width and the packing  $N_{\text{op}}$ ) and on where the operands reside (Horowitz, 2014). The supply voltage  $V$  tracks the voltage–frequency operating point (Palipadi & Starikovskiy, 2006). Every one of them is under the prover’s control, and none is visible to an off-chip meter. Empirically the spread is wide: component-level analyses of a 45 nm process put arithmetic and on-chip memory access at 0.1–20 pJ per operation depending on format and operand locality (Horowitz, 2014).

In practice, the meter reports only a noisy version of the power,

$$P_{\text{obs}}(t) = P(t) + \eta(t), \quad (3)$$

with  $\eta(t)$  a zero-mean measurement noise the prover cannot forge. The noise is a measurement limit: zero-mean noise with a suitable correlation structure (stationary, finitely correlated) averages away over the window and shrinks the longer the verifier watches. Throughout this paper we therefore assume a perfect meter and work with the clean power  $P(t)$ ; every floor we derive is a floor on what even a noiseless meter can establish.

We count hardware operations (i.e. what the silicon actually executes) rather than model FLOPs (i.e. the algorithmically required operations). Every executed operation costs joules, so the meter sees hardware operations. We count them as nominal operations of whichever numeric format runs: a multiply–add retires two nominal ops whether it executes at fp32 or int8, and integer multiply–adds are ops, not FLOPs.

### 2.2. The threat model

There are two parties. The *prover* is the monitored operator: it controls the workload, the operand values, the numeric formats, the clocks, and the idle states of the machine, and it may wish to run compute it does not declare. The *verifier* is the inspecting party: it reads  $P_{\text{obs}}$  from an off-chip meter it trusts, and it never sees  $r$ ,  $F$ , or  $P_0$  separately. The prover declares a workload of  $C_{\text{dec}}$  operations. The verifier wants to know if  $C_{\text{dec}}$  is consistent with  $P_{\text{obs}}$ . The verifier makes two assumptions:

**Assumption 2.1** (Trusted meter). The power trace cannot be forged:  $P_{\text{obs}}$  differs from the true draw  $P$  only by zero-mean noise, which we set to zero throughout.

**Assumption 2.2** (Trusted declaration). The declared computation really ran: the smallest total consistent with the window is  $C_{\text{dec}}$  itself. A power meter cannot by itself confirm this; independently checking that the declared work ran is a separate verification problem—the domain of proof-

of-learning and zero-knowledge proof-of-training protocols (Jia et al., 2021; Garg et al., 2023).

Everything else is under the prover’s control and unobserved: the exchange rate  $r(t)$ , the overhead  $P_0(t)$ , and how the operation rate  $F(t)$  splits between declared and covert work.

### 2.3. The inverse model

Define  $G$  as the forward map that carries a workload to its power trace,

$$G : (F(\cdot), r(\cdot), P_0(\cdot)) \mapsto P(\cdot), \quad (4)$$

defined pointwise by Equation (2). The verifier faces an inverse problem: read the power trace and recover the workload that produced it, i.e. invert  $G$ . But  $G$  is *non-injective*. Since  $r(\cdot)$  is free to vary, a low rate spent on much computation draws exactly the same power as a high rate spent on little. Distinct workloads  $(F, r, P_0)$  map to identical traces, and a many-to-one map has no inverse. Inversion therefore returns the identified set  $\mathcal{A}(P_{\text{obs}})$  which collects every physically admissible workload that reproduces  $P_{\text{obs}}$  (Figure 1).

The question “how many operations ran over the window?” corresponds to one specific projection of  $\mathcal{A}(P_{\text{obs}})$ . Define the total compute

$$C = \int_0^T F dt \quad [\text{op}]. \quad (5)$$

Reading  $C$  off each admissible workload projects the identified set onto the  $C$ -axis,

$$\{C : (F, r, P_0) \in \mathcal{A}(P_{\text{obs}})\} = [C_{\text{lo}}, C_{\text{hi}}]. \quad (6)$$

We define  $\beta$  as the width of that interval, measured against the machine’s peak declared-format capacity  $C_{\text{max}} = F_{\text{max}}^{\text{dec}} T$ ,

$$\beta \equiv \frac{C_{\text{hi}} - C_{\text{lo}}}{C_{\text{max}}}. \quad (7)$$

Under Assumption 2.2 the lower edge is the declaration itself,  $C_{\text{lo}} = C_{\text{dec}}$ . The interval runs from that honest baseline up to the most compute the meter cannot rule out, so the upper edge is the declared work plus the largest covert load a cheater could slip past the meter,

$$C_{\text{hi}} = C_{\text{dec}} + C_{\text{cov}}^{\text{max}}. \quad (8)$$

Substituting  $C_{\text{lo}} = C_{\text{dec}}$  and Equation (8) into Equation (7) leaves a single unknown,

$$\beta = \frac{C_{\text{hi}} - C_{\text{lo}}}{C_{\text{max}}} = \frac{C_{\text{cov}}^{\text{max}}}{C_{\text{max}}}. \quad (9)$$

The width  $C_{\text{hi}} - C_{\text{dec}} = C_{\text{cov}}^{\text{max}}$  is exactly the covert operations a cheater could hide. A wider interval means a larger  $\beta$  and more room for undeclared work;  $\beta \gg 1$  means the ambiguity exceeds the entire machine.

## 3. Derivation of $\beta$

We now derive a closed form for  $\beta$ . Two physical constraints cap the covert work that can run inside the observation window  $[0, T]$ . The first is the *energy budget*: the total the meter reports must stay explainable as honest declared work, so covert operations can spend only the joules that ambiguity leaves free (Section 3.1). The second is *capacity*: covert operations must also fit on the machine, in whatever machine-time the declared work leaves unused (Section 3.2). The floor  $\beta$  is the smaller of the two widths (Section 3.3).

### 3.1. The energy budget

The derivation of the first width is straightforward: integrate the forward model over the window, split the total energy into declared, covert, and overhead pieces, and ask how large the covert piece can grow before the total stops looking honest. We work with the integrated trace (i.e. the total energy) rather than the resolved time series: treating power as affine in the operation rate (Equation (2)) lets its integral depend on the workload only through time-integrated quantities.

Integrating the forward model Equation (2) over  $[0, T]$ , the left-hand side is the total energy the meter reports and the right-hand side falls into two pieces,

$$E = \underbrace{\int_0^T r(t) F(t) dt}_{\text{compute energy } E_c} + \underbrace{\int_0^T P_0(t) dt}_{\text{overhead } E_0}. \quad (10)$$

The overhead  $E_0$  is a nuisance the verifier knows only within a band  $E_0 \in [E_{0,\text{lo}}, E_{0,\text{hi}}]$ . We do not “subtract off” a known baseline: the prover controls fans, clocks, and idle states, so a known-and-removed  $P_0$  would hand it a hiding channel.

The compute energy  $E_c$  splits into a declared block and a covert block,

$$\int_0^T r(t) F(t) dt = \underbrace{\int_0^T r^{\text{dec}}(t) F_{\text{dec}}(t) dt}_{\text{declared}} + \underbrace{\int_0^T r^{\text{cov}}(t) F_{\text{cov}}(t) dt}_{\text{covert}}. \quad (11)$$

The declared block runs  $C_{\text{dec}} = \int_0^T F_{\text{dec}} dt$  operations, taken known, and the covert block runs the excess  $C_{\text{cov}} = C - C_{\text{dec}}$ ; this is the compute we are trying to bound. Write each block’s energy as its operation count times a single mean cost per operation, i.e. the op-weighted averages

$$\bar{r}^i \equiv \frac{1}{C_i} \int_0^T r^i(t) F_i(t) dt = \int_0^T r^i(t) \frac{F_i(t)}{C_i} dt, \quad (12)$$

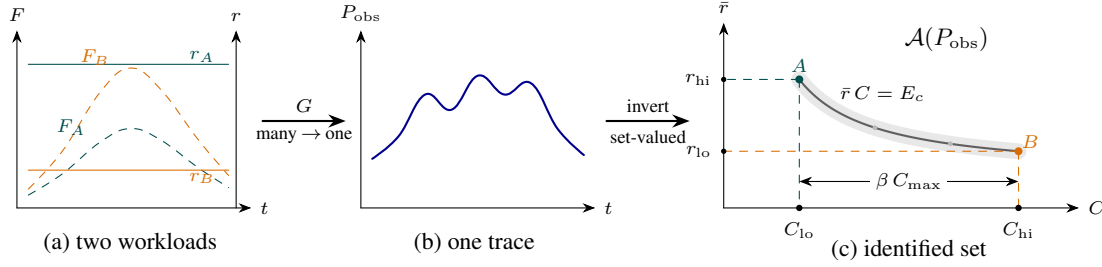


Figure 1. Non-injectivity of the forward map. (a) Two workloads (A little compute at a high rate, B much compute at a low rate) whose products  $rF$  coincide, so (b) they drive the identical power trace. (c) Inverting recovers only the set of consistent explanations  $\mathcal{A}(P_{\text{obs}})$ : the rate band  $[r_{\text{lo}}, r_{\text{hi}}]$  cuts the constant-energy arc  $A-B$ , whose projection onto the  $C$ -axis spans  $[C_{\text{lo}}, C_{\text{hi}}]$ ; relative to capacity that width is the floor  $\beta$ . The faint grey ribbon is meter noise (ignored in this work). Each rate is drawn constant only for clarity; in reality  $r(t)$  varies in time. This does not matter for a question about total operations, which depends on  $r(t)$  only through its op-weighted mean  $\bar{r} = E_c/C$  (compute energy  $E_c = \int_0^T rF dt$  over total operations  $C$ ). That mean is panel (c)'s vertical axis, so letting  $r(t)$  vary leaves the picture unchanged.

for  $i \in \{\text{dec}, \text{cov}\}$ . The second form makes the averaging explicit: the weight  $F_i(t) dt/C_i$  is the fraction of the block's operations that run in  $[t, t + dt]$ ; it is nonnegative and integrates to one. The machine therefore draws

$$E = \underbrace{\bar{r}^{\text{dec}} C_{\text{dec}}}_{\text{declared work}} + \underbrace{\bar{r}^{\text{cov}} C_{\text{cov}}}_{\text{covert work}} + \underbrace{E_0}_{\text{overhead}}, \quad (13)$$

and the verifier sees only  $E$ .

The two mean costs are bounded,  $\bar{r}^{\text{dec}} \in [r_{\text{lo}}^{\text{dec}}, r_{\text{hi}}^{\text{dec}}]$  and  $\bar{r}^{\text{cov}} \geq r_{\text{lo}}^{\text{cov}}$ .<sup>1</sup> These band edges,

$$(r_{\text{hi}}^{\text{dec}}, r_{\text{lo}}^{\text{dec}}, r_{\text{lo}}^{\text{cov}}), \quad (14)$$

are the whole parameter vector of this paper: from here on, all that changes is which setting of the triple a verifier can justify.

The verifier raises no alarm so long as the measured total is explainable by honest declared work at some admissible rate and overhead. The most an honest run could ever draw is the declared work at the top of its band, on top of the highest overhead,

$$E \leq r_{\text{hi}}^{\text{dec}} C_{\text{dec}} + E_{0,\text{hi}}. \quad (15)$$

The test is one-sided because covert work can only add energy: only a total that runs high is suspicious. Substituting the budget Equation (13) into the no-alarm condition Equation (15) and rearranging,

$$C_{\text{cov}} \leq \frac{(r_{\text{hi}}^{\text{dec}} - \bar{r}^{\text{dec}}) C_{\text{dec}} + (E_{0,\text{hi}} - E_0)}{\bar{r}^{\text{cov}}}. \quad (16)$$

A cheater now pushes every free choice on the right-hand side to its limit:

<sup>1</sup>No upper edge is needed for the covert cost:  $\bar{r}^{\text{cov}}$  enters the bound only as a divisor in Equation (16), so the worst case sits at its lower edge and a ceiling would never bind—expensive covert work only hurts the adversary.

- *Run the declared work cheaply*,  $\bar{r}^{\text{dec}} \rightarrow r_{\text{lo}}^{\text{dec}}$ . The verifier cannot tell an expensive declared run from a cheap one, so it must allow the draw all the way up to  $r_{\text{hi}}^{\text{dec}} C_{\text{dec}}$ ; the gap  $(r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}}) C_{\text{dec}}$  is free energy the prover spends covertly while the total still reads as honest work.
- *Idle the overhead*,  $E_0 \rightarrow E_{0,\text{lo}}$ , freeing the full band width  $\Delta E_0 = E_{0,\text{hi}} - E_{0,\text{lo}}$ .
- *Run the covert work cheaply*,  $\bar{r}^{\text{cov}} \rightarrow r_{\text{lo}}^{\text{cov}}$ , so each hidden operation costs as little as the hardware allows.

At those values Equation (16) attains its maximum, the largest covert load the energy budget admits,

$$C_{\text{cov}}^{\text{max, en}} = \frac{(r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}}) C_{\text{dec}} + \Delta E_0}{r_{\text{lo}}^{\text{cov}}}. \quad (17)$$

Divide by  $C_{\text{max}}$  and write the declared hardware utilisation  $h = C_{\text{dec}}/C_{\text{max}}$  for the fraction of the machine's declared-format capacity the declared work occupies. The result is a covert load in the normalised units of Equation (7): the width the identified set would have if energy were the only constraint. We write it  $\beta_{\text{en}}$ , the *energy-only width*,

$$\beta_{\text{en}}(h) = h \frac{r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}}}{r_{\text{lo}}^{\text{cov}}} + \frac{\Delta E_0}{r_{\text{lo}}^{\text{cov}} C_{\text{max}}}. \quad (18)$$

Pedagogically, Equation (18) describes (how much honest work there is)  $\times$  (how uncertain its per-operation cost is), divided by (how cheaply covert work can run), plus a smaller slack from the overhead band. The first term grows with the declared-band width and with the declared utilisation: more legitimate work offers more cover. The second term is the overhead's contribution: joules the prover frees by idling overhead that the verifier must still allow for.

Note the asymmetry between the numerator and denominator of Equation (18). Declarations and re-execution can

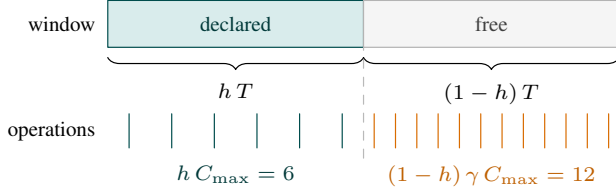


Figure 2. The capacity clip. Declared work takes  $hT$  of the window and the covert block gets the rest (top); each block executes operations at its own format’s rate (bottom), drawn at  $h = 1/2$  and  $\gamma = 2$  (int8 covert work against declared fp16). The free half-window hides a full  $C_{\max} = 12$  worth of covert work.

narrow the declared band width  $r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}}$  in the numerator, but no declaration reaches the covert floor  $r_{\text{lo}}^{\text{cov}}$  in the denominator; only hardware-wide observables (clock, voltage) can raise  $r_{\text{lo}}^{\text{cov}}$ . The overhead width  $\Delta E_0$  is a second numerator, and no declaration reaches it either: it is freedom the prover exercises inside the window rather than ignorance about a quantity fixed during it, so characterising it off-site measures it without shrinking it. It yields only to an on-site observable of the idle state, or to a restriction on which overhead states may accompany declared work (Section 5).

### 3.2. The capacity clip

The second constraint is capacity: covert work must also fit on the machine. The declared and covert streams multiplex one device across a single window of length  $T$ , so the covert block runs only in the time the declared block leaves free. What turns free time into an operation count is a rate. The realised rate  $F(t)$  is the prover’s to choose, so a worst-case bound must price the time at the one rate the verifier does know: the published peak  $F_{\max}$ . But the peak is not just a property of the hardware; it is also a property of the executing format. An A100’s dense peaks run from 19.5 Top/s at fp32 on the CUDA cores through 156 at tf32 and 312 at fp16/bf16 to 624 Top/s at int8 on the tensor cores (NVIDIA Corporation, 2020). We define the capacity ratio of the covert format against the declared one,

$$\gamma = \frac{F_{\max}^{\text{cov}}}{F_{\max}^{\text{dec}}} \quad (19)$$

( $\gamma = 2$  for int8 covert work against fp16 declared work on an A100). The declared work of  $C_{\text{dec}} = h C_{\max}$  operations cannot be executed in less than  $C_{\text{dec}}/F_{\max}^{\text{dec}} = hT$  of the window, so at most  $(1-h)T$  is left free (Figure 2). Run at the covert format’s peak, that free time is worth  $F_{\max}^{\text{cov}} (1-h)T$  covert operations. Dividing by  $C_{\max} = F_{\max}^{\text{dec}} T$  gives

$$C_{\text{cov}}/C_{\max} \leq (1-h)\gamma. \quad (20)$$

When  $\gamma > 1$  the covert format executes more operations per unit of machine-time than the declared one, so  $C_{\text{dec}} + C_{\text{cov}}$  may exceed  $C_{\max}$ : the normaliser is the machine’s capacity

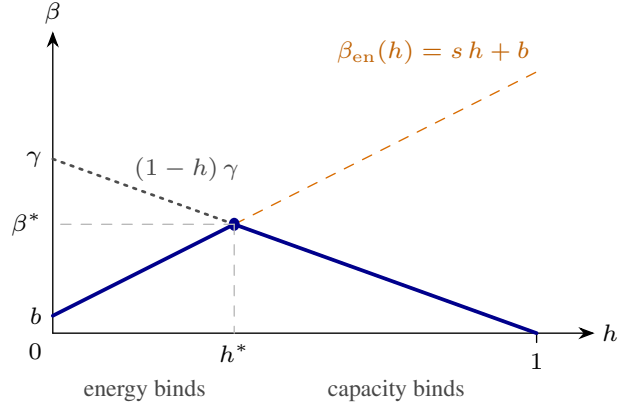


Figure 3. The shape of the floor. The energy-only width  $\beta_{\text{en}}(h) = sh + b$  rises with declared utilisation—more honest work offers more cover (dashed)—while the capacity clip  $(1-h)\gamma$  falls, since more honest work leaves less machine free (dotted). The floor Equation (21) is the lower of the two (solid): a tent peaking at the crossing  $(h^*, \beta^*)$  of Equation (22), with energy binding to its left, capacity to its right, and  $\beta(1) = 0$  because a fully occupied machine leaves nowhere to hide.

priced in *declared-format* operations, not a ceiling on how many operations can run.

### 3.3. The floor

The covert load must fit under both caps at once, so the floor is the smaller of the two widths,

$$\beta(h) = \min[\beta_{\text{en}}(h), (1-h)\gamma]. \quad (21)$$

The two caps pull in opposite directions (Figure 3). Energy rises with  $h$ —more declared work offers more cover—while capacity falls—more declared work leaves less machine to hide on. At full declared utilisation the machine is spoken for and  $\beta(1) = 0$ ; at idle there is capacity but almost no cover. The declared utilisation is the prover’s to set, so the guarantee a verifier can quote is the worst case over it,  $\beta^* \equiv \max_h \beta(h)$ ; this is the figure of merit the rest of the paper tracks. The maximum sits at the crossing,

$$h^* = \frac{\gamma - b}{s + \gamma}, \quad \beta^* = \gamma \frac{s + b}{s + \gamma}, \quad (22)$$

writing  $\beta_{\text{en}}(h) = sh + b$  for its slope and intercept. The crossing form holds when  $b < \gamma$ ; if the overhead term alone exceeds the clip ( $b \geq \gamma$ ) the clip binds everywhere and the maximum sits at the boundary,  $(h^*, \beta^*) = (0, \gamma)$ . Each side of the minimum is the binding constraint in its own regime: energy binds for  $h < h^*$  (the trace itself caps the covert load), capacity binds for  $h > h^*$ , and both bind at the crossing. Where the crossing sits is an empirical question about  $s$ .

One caveat is that the cheating strategy of Section 3.1

pushed three quantities to their edges at once—cheap declared work, idled overhead, cheap covert work—and no single schedule is guaranteed to reach all three edges together, so  $\beta$  upper-bounds the true identified-set width. Appendix B states this and discusses this corner assumption precisely.

#### 4. How big is $\beta$ ?

We now estimate the typical value of  $\beta$ , Equation (21), for real hardware. We first ask what physically drives each entry of the triple Equation (14) (Section 4.1), then measure them directly on an NVIDIA A100, where each experiment is built to pin one entry (Section 4.2). We focus on the  $\beta_{\text{en}}(h)$  component, Equation (18): it carries all the physics, whereas the clip Equation (20) is a ratio of published peaks with nothing to measure. The capacity clip Equation (20) returns at the end, when the measured bands are assembled.

##### 4.1. The two terms

The energy-only width  $\beta_{\text{en}}(h)$  is a sum of two terms: an ambiguity term  $s h$  with slope  $s = (r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}})/r_{\text{lo}}^{\text{cov}}$ , set by the spread of the declared exchange rate against the price of a covert operation, and an overhead term  $b = \Delta E_0/(r_{\text{lo}}^{\text{cov}} C_{\text{max}})$ , set by the width of the overhead band against the same price. We take each term in turn.

**The exchange-rate term.** Section 2.1 wrote the exchange rate as  $r = \alpha (C/N_{\text{op}}) V^2$ . Here we split the capacitance term in two, since the numeric format and the operand locality move independently:

$$r \approx \underbrace{\kappa}_{\text{precision}} \cdot \underbrace{\alpha}_{\text{operands}} \cdot \underbrace{C_{\text{eff}}}_{\text{locality, kernel}} \cdot \underbrace{V^2}_{\text{DVFS}}. \quad (23)$$

Appendix A derives this factorisation. Each factor is set by a different physical scale of the machine (Figure 4). The numeric-format scale  $\kappa$  decreases with datapath width at each precision step (fp32  $\rightarrow$  int8), tracking the pJ-scale energy hierarchy of arithmetic (Horowitz, 2014); the activity factor  $\alpha$  is the data-dependent fraction of transistors that toggle for the given operands; the effective capacitance  $C_{\text{eff}}$  grows as operands become less local, because more of the memory hierarchy (e.g. storage arrays and peripheral logic, buffers and interfaces, interconnect) switches per operation (Horowitz, 2014; Chatterjee et al., 2017); and  $V^2$  is the capacitor-charging energy of the supply voltage, which DVFS slides along the voltage–frequency curve (Pallipadi & Starikovskiy, 2006).

With the power meter as the only channel, every factor of Equation (23) is conceded at once. If each ranges over an interval the verifier cannot narrow, and the factors can be extreme together, the declared band’s edges are the products

of the factor edges,

$$r_{\text{hi}}^{\text{dec}} = \kappa_{\text{hi}} \alpha_{\text{hi}} C_{\text{hi}} V_{\text{hi}}^2, \quad r_{\text{lo}}^{\text{dec}} = \kappa_{\text{lo}} \alpha_{\text{lo}} C_{\text{lo}} V_{\text{lo}}^2, \quad (24)$$

writing  $C$  for  $C_{\text{eff}}$  to keep the subscripts readable. Dividing, the band’s ratio is the product of per-factor widths

$$\frac{r_{\text{hi}}^{\text{dec}}}{r_{\text{lo}}^{\text{dec}}} = w_{\kappa} w_{\alpha} w_C w_V, \quad w_x \equiv \frac{x_{\text{hi}}}{x_{\text{lo}}}, \quad w_V \equiv \frac{V_{\text{hi}}^2}{V_{\text{lo}}^2}, \quad (25)$$

each  $w_x \geq 1$ . Taking the suprema together makes this an outer bound, for the same reason the caveat closing Section 3.3 gave for the floor’s own corner; Appendix B treats both. Writing  $W \equiv w_{\kappa} w_{\alpha} w_C w_V$  we can express the slope as

$$s = \frac{r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}}}{r_{\text{lo}}^{\text{cov}}} = \frac{r_{\text{lo}}^{\text{dec}}}{r_{\text{lo}}^{\text{cov}}} (W - 1). \quad (26)$$

Note we never evaluate  $\beta$  from the factorisation; every number we report comes from measured band edges directly.

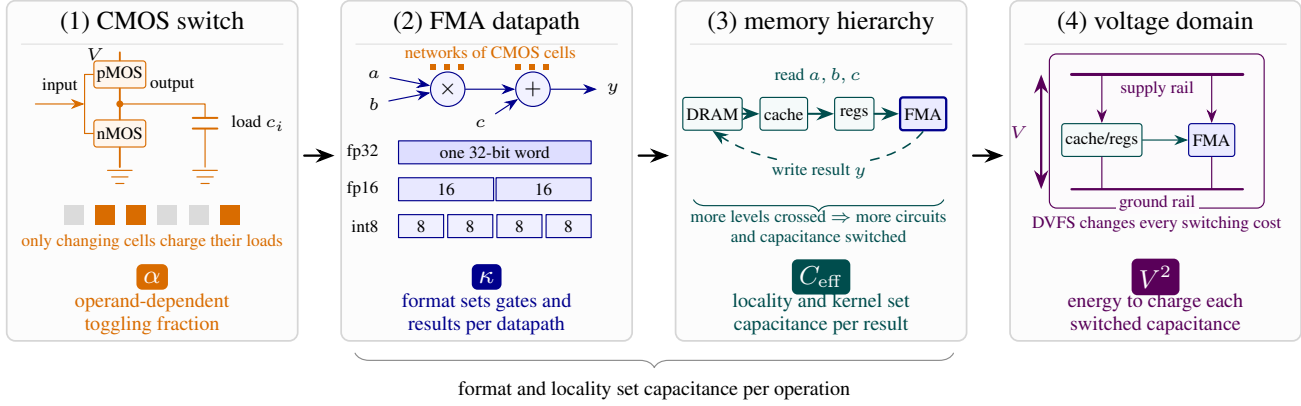
**The overhead term.** The overhead  $P_0$  admits no comparably clean factorisation. It is a heterogeneous mix of static leakage, cooling control loops, firmware idle states, and platform I/O, so no single relation fixes the band width  $\Delta E_0$  from first principles.

##### 4.2. Measurement on an NVIDIA A100

In order to measure  $\beta$  we require estimates of the three entries of the triple  $(r_{\text{hi}}^{\text{dec}}, r_{\text{lo}}^{\text{dec}}, r_{\text{lo}}^{\text{cov}})$ , Equation (14), and the overhead width  $\Delta E_0$ . Each experiment below exists to pin one of them. Experiment 1 supplies both declared edges  $(r_{\text{hi}}^{\text{dec}}, r_{\text{lo}}^{\text{dec}})$ , Experiment 2 supplies  $\Delta E_0$ , and Experiment 3 supplies the covert floor  $r_{\text{lo}}^{\text{cov}}$ . Results are summarised in Table 1.

**Setup and provenance.** The experiments below ran sequentially on one NVIDIA A100-SXM4-80GB in a single Slurm allocation. The platform denied clock locking and power capping, so DVFS is a recorded axis rather than a controlled one: each window logs the streaming-multiprocessor (SM) clock selected by the driver’s frequency governor, and the bands are stratified into clock strata. That also puts  $V^2$  beyond measurement here, so of the four factors of Equation (23) we measure three and carry  $w_V \approx 1.5$  from device physics throughout; this is discussed as a limitation in Section 6.2. Energy is read from the on-die NVML total-energy counter; each measured cell fills a  $\sim 1.5$  s window with back-to-back kernels on pre-allocated buffers and keeps the per-cell median over repeated windows. This is a non-adversarial, device-level calibration: the telemetry is trusted

<sup>2</sup>Supply voltages on modern accelerators sit near 1 V (Horowitz, 2014) and DVFS moves them by roughly  $\pm 10\%$ ; energy per operation scales as the ratio squared,  $(1.1/0.9)^2 \approx 1.5$ .



$$r = \alpha \cdot \kappa \cdot C_{\text{eff}} \cdot V^2$$

$f$  cancels from the marginal rate; leakage and cooling leave as the overhead  $P_0$ , not  $r$

**Figure 4. The exchange rate, from transistors to the powered die.** Each physical scale contributes one factor to the per-operation energy of Equation (23). (1) A complementary pMOS–nMOS inverter charges or discharges its effective load  $c_i$  when its input changes; an operand determines what fraction  $\alpha$  of all such cells toggle. (2) Networks of these cells implement the multiply and add of an FMA. The numeric format selects the word width (and supported packing), changing the logic engaged per arithmetic result and hence  $\kappa$ . (3) The FMA reads operands through the register–cache–DRAM hierarchy and writes its result back. Traversing more of that hierarchy switches additional array and peripheral circuits, buffers and interfaces, and interconnect; their combined dynamic cost is represented by  $C_{\text{eff}}$ . (4) Within a voltage domain, the supply places  $V$  across the arithmetic and interconnect loads, so DVFS contributes  $V^2$  per unit capacitance. The clock  $f$  cancels from the marginal rate (Equation (39)) and the static draw—leakage, cooling—leaves as the overhead  $P_0$ . Multiplying the factors recovers Equation (23).

as a laboratory instrument. One GPU suffices to estimate the bands and the order of magnitude of  $\beta$ ; extending the calibration to off-chip meters and multi-device systems is natural future work, taken up in Section 6. Appendix F gives the full protocol, the meter tolerance, and the provenance manifest, including which card measured which band.

**Experiment 1—the declared band** ( $r_{\text{hi}}^{\text{dec}}, r_{\text{lo}}^{\text{dec}}$ ), **and three of the four factor widths** ( $w_\kappa, w_\alpha, w_C$ ). A compute-bound matmul (operation count  $C = 2MNK$  known analytically<sup>3</sup>), swept factorially over precision ( $\kappa$ , fp32 down to int8), operand locality ( $C_{\text{eff}}$ , operands resident in vs. streamed through the L2 at a byte-identical shape), and operand distribution ( $\alpha$ , zeros through adversarial-toggle).

The floor’s two declared entries come off that sweep directly. With no format pinned they are its extremes, the most expensive cell (fp32, adversarial-toggle operands) against the cheapest (int8, zeros; Figure 5),

$$r_{\text{hi}}^{\text{dec}} = 19.5 \text{ pJ/op}, \quad r_{\text{lo}}^{\text{dec}} = 0.786 \text{ pJ/op}. \quad (27)$$

The sweep is a factorial so the widths of Equation (25) are also obtained for free. The 60 cells are every combination of five precisions, two localities and six operand distributions. To measure one factor’s width, hold the other two axes fixed

<sup>3</sup>The int8 cells retire integer multiply–adds, counted as nominal operations per the convention of Section 2.1

and take the ratio of the most to the least expensive cell as that factor runs over all its levels; the width is the median of that ratio across every such group of cells. This gives

$$w_\kappa = 13.5, \quad w_\alpha = 1.88, \quad w_C = 1.04, \quad (28)$$

with precision following the textbook ordering  $\text{int8} < \text{bf16} \approx \text{fp16} < \text{tf32} < \text{fp32}$ . So precision dominates, operand values are worth a factor near two, and locality moves  $r$  by four per cent for this compute-bound kernel.<sup>4</sup> The clock is the one axis we could not hold fixed, so we check it rather than group on it. Comparing the median achieved clock across each axis’s levels, the operand and locality levels ran at an identical clock and the precision levels to within 2.7%, so these widths are not clock artefacts. For the same reason the fourth width is missing: the governor held the clock inside 1335–1410 MHz across the whole sweep, a  $\pm 3\%$  window, which is what keeps the three measured widths clean but leaves  $w_V$  no range to be measured over.

Multiplying the three measured widths gives  $13.5 \times 1.88 \times 1.04 = 26.5$ , against the  $24.8\times$  span the pooled cells actually show—so Equation (25) holds to 7%, the residual

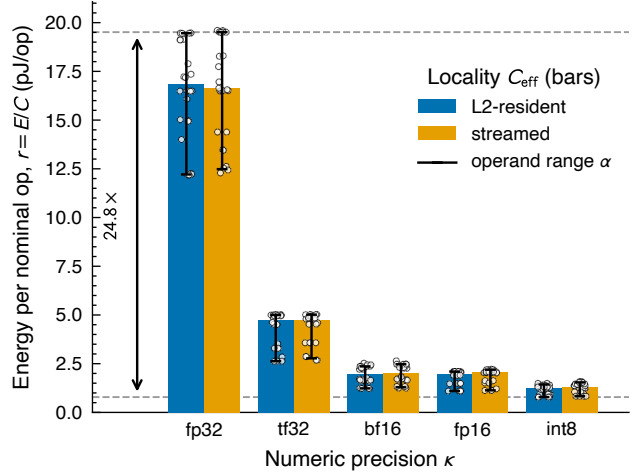
<sup>4</sup>Per access, DRAM costs about a hundred times more than on-chip memory (Horowitz, 2014); the per-operation effect is diluted by the kernel’s bytes-per-operation, which is why  $w_C$  is small here. A memory-bound kernel would not enjoy that dilution, and  $w_C$  is the one width we would expect to move substantially with the workload.

| quantity                                             | what it is               | source             | value               |
|------------------------------------------------------|--------------------------|--------------------|---------------------|
| <i>(a) the factor widths of Equation (25)</i>        |                          |                    |                     |
| $w_\kappa$                                           | precision                | Exp. 1, cond.      | $13.5\times$        |
| $w_\alpha$                                           | operands                 | Exp. 1, cond.      | $1.88\times$        |
| $w_C$                                                | locality                 | Exp. 1, cond.      | $1.04\times$        |
| $w_V$                                                | clock (DVFS)             | not swept; assumed | $\approx 1.5\times$ |
| $w_\kappa w_\alpha w_C$                              | product                  | —                  | $26.5\times$        |
| <i>(b) the edges the floor reads, nothing pinned</i> |                          |                    |                     |
| $r_{hi}^{dec}$                                       | fp32, expensive operands | Exp. 1             | 19.5                |
| $r_{lo}^{dec}$                                       | int8, zeros              | Exp. 1             | 0.786               |
| $r_{lo}^{cov}$                                       | generic covert op        | Exp. 3             | 0.51                |
| $\Delta P_0$                                         | overhead band            | Exp. 2             | 41.2 W              |

**Table 1.** Summary of results from Experiments 1–3. Values in pJ/op unless marked otherwise. Block (a) is the physics: the four factors of Equation (23) and how wide each one runs. “Cond.” means the width is measured with the other three factors held fixed, so the widths are separable rather than pooled. Their product reproduces the measured span to 7%, and  $w_\alpha w_C = 1.96$  predicts the fp16 sub-band’s measured 2.00 to 2%.  $w_V$  is the exception and is flagged as such: Exp. 1 does not sweep the clock (the governor held it inside a  $\pm 3\%$  window), so its width is a physics estimate, not a measurement. Block (b) is what the floor reads: the triple Equation (14) and the overhead width. These are the values a bare power meter concedes, with no format pinned on either declared edge and the covert floor priced for generic covert work.  $C_{max}$  and  $\gamma$  are read from published peaks rather than measured here.

being factor interaction. Similarly, among the fp16 cells the precision never varies, so the spread should be the product of the two remaining widths,  $w_\alpha w_C = 1.96$ ; those cells span  $[1.10, 2.19]$  pJ/op, a factor of 2.00. Appendix G performs a separate fp16-only sweep of the operand axis with a governor-selected clock and corroborates  $w_\alpha$  independently, reaching 1.83 compared to the quoted value here of 1.88.

**Experiment 2—the overhead band  $\Delta P_0$  (hence  $\Delta E_0$ ).** The overhead is what the meter reads with no declared work running. We measure five variants chosen to bracket the plausible range: before any CUDA context, context up, context plus operand pool resident, immediately after a sustained heavy kernel, and after a 60 s cooldown. Each variant is measured over five repeated windows. The band is the lowest and highest of those 25 windows, not a range over per-variant medians. The lowest is a cold card before any context, at 62.9 W. The highest is a window taken immediately after the heavy kernel, at 96.2 W. That variant is a transient rather than a steady state: its five consecutive windows decay from 96.2 to 76.5 W as the card settles. We take its peak deliberately. An adversary can leave a device in that state, and the meter does not report which idle state it is reading, so the peak belongs in the envelope. Reducing each variant to a median instead would give  $\Delta P_0 = 23.9$  W. The observed range  $[62.9, 96.2]$  W, stretched outward by



**Figure 5.** Experiment 1: Total energy quotient  $E/C$  for a fixed matmul kernel on an NVIDIA A100, swept factorially over precision  $\kappa$  (bar groups), operand locality  $C_{eff}$  (paired bars), and operand distribution  $\alpha$  (whiskers); white points are the individual measured windows ( $n = 240$  across 60 cells). Each bar is the median over the operand axis and its whisker that axis’s full range, so the operand factor (a little under  $2\times$  inside every format, against locality’s couple of percent) is read off the figure rather than pooled into it. Dashed lines mark the widest setting’s edges, both of them operand extremes: fp32 at its most expensive operands, int8 with zeros.

the adopted  $\pm 5\%$  meter tolerance<sup>5</sup>, pins the overhead band to

$$P_0 \in [59.8, 101.0] \text{ W}, \quad \Delta P_0 = 41.2 \text{ W}. \quad (29)$$

**Experiment 3—the marginal covert cost  $r_{lo}^{cov}$ .** We construct a 10 s observation window. Inside it we run a fixed fp16 declared burst—a 2048<sup>3</sup> matmul on declared-typical operands, repeated enough times to keep the device busy for 30% of the window—then  $N_{cov}$  covert matmuls of a single format (int8 or fp16, each swept as its own arm), then an idle pad that fills out the remainder. The covert operands are zeros, the adversary’s cheapest case. The two formats bracket what matters: int8 is the cheapest arithmetic the device offers, and fp16 matches the declared work, so covert operations are indistinguishable from declared ones by format alone. Only  $N_{cov}$  changes between windows: the declared burst, the shapes, the operand buffers, and the 10 s length are all held fixed.

<sup>5</sup>NVML publishes no accuracy specification for the total-energy counter `nvmlDeviceGetTotalEnergyConsumption` on GA100; the documented  $\pm 5\%$  is for the power readings of the earlier Fermi and Kepler generations (NVIDIA Corporation, 2026). We adopt it as a working tolerance and apply it outward only, so it can only widen  $\beta$  (Appendix F). It accounts for 7.9 W of the width below; the remaining 33.3 W is genuine spread between idle states.

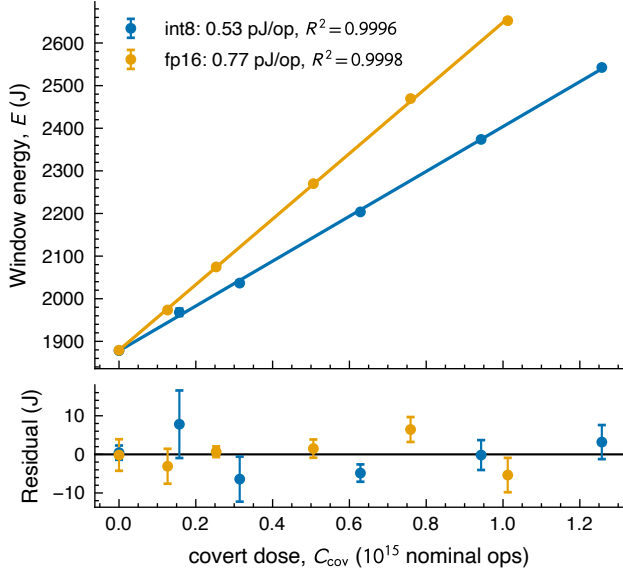


Figure 6. Experiment 3: window energy  $E$  vs. covert operation count  $C_{\text{cov}}$  for int8 and fp16 covert work, added to a fixed fp16 declared burst inside a fixed 10 s window. *Top*: per-cell medians with  $\pm 1$  SD error bars over each cell’s four untrimmed repeats (of five captured; the flagged high-tail repeat is excluded), against the fit  $E = r_{\text{cov}} C_{\text{cov}} + E_0$ ; both slopes are linear to  $R^2 > 0.999$  (0.53 pJ/op int8, 0.77 pJ/op fp16) and share an intercept matching the directly measured covert-free windows to 0.02%. *Bottom*: fit residuals.

To see how this experiment determines  $r_{\text{lo}}^{\text{cov}}$ , write the window energy out. Let  $e_{\text{cov}}$  be the energy of one covert matmul,  $t_{\text{cov}}$  its duration,  $c$  its nominal operation count,  $t_{\text{dec}}$  the declared burst’s busy time, and  $P_{\text{idle}}$  the power drawn during the pad. Then

$$E = E_{\text{dec}} + N_{\text{cov}} e_{\text{cov}} + P_{\text{idle}}(T - t_{\text{dec}} - N_{\text{cov}} t_{\text{cov}}), \quad (30)$$

which collects into a constant plus a term linear in  $N_{\text{cov}}$ ,

$$E = \underbrace{E_{\text{dec}} + P_{\text{idle}}(T - t_{\text{dec}})}_{\text{independent of } N_{\text{cov}}} + N_{\text{cov}} \underbrace{(e_{\text{cov}} - t_{\text{cov}} P_{\text{idle}})}_{\text{the slope}}. \quad (31)$$

The first bracket does not depend on  $N_{\text{cov}}$ . The covert work amounts to  $C_{\text{cov}} = N_{\text{cov}} c$  operations, so differentiating leaves only the second,

$$\frac{\partial E}{\partial C_{\text{cov}}} = \frac{e_{\text{cov}} - t_{\text{cov}} P_{\text{idle}}}{c} \equiv r^{\text{cov}}. \quad (32)$$

We sweep  $N_{\text{cov}}$  over six levels, from no covert work up to a covert busy fraction of 0.48, with five repeats per level in randomised order with drift-reference splices. Regressing window energy on covert nominal operations then gives the marginal cost directly (Figure 6): 0.53 pJ/op for int8 and 0.77 pJ/op for fp16, both linear to  $R^2 > 0.999$ . Pricing the same covert kernels the obvious way instead—total energy

divided by total operations—gives 0.26 (int8) to 0.39 pJ/op (fp16) more. That excess is baseline power charged to the covert operations, which is what the fixed window differences away. We take the one-sided 95% lower confidence limits,

$$r_{\text{lo}}^{\text{cov}} = 0.51 \text{ pJ/op (int8)}, \quad 0.76 \text{ pJ/op (fp16)}. \quad (33)$$

Because the covert cost divides the slack, under-pricing covert work is the security-safe direction.

One caveat is that, similar to the other experiments, clock locking is denied on this platform, so the governor is recorded rather than controlled here too. More covert work leaves a shorter idle pad, and the window-median clock rises with the load accordingly, from 1275 MHz with no covert work to 1410 MHz at the heaviest. However what changes is the time spent at each clock frequency, not the operating point the kernels run at: every measured window, in both arms, records the same 1275 MHz minimum and 1410 MHz maximum. We would rather have locked the clock and compared every cell at one fixed setting. Two checks stand in for that. A clock inflating the cost of later operations would bend the response, and the fits show no systematic curvature; and the zero end of each fit lands on windows we measured directly with no covert work at all (Figure 6). We record the missing clock control as a limitation of the platform (Section 6).

**Assembling  $\beta$ .** We now combine the three experiments into an estimate of  $\beta$  (cf. Table 1):

- the declared band, from Experiment 1’s fp16 cells:  $[r_{\text{lo}}^{\text{dec}}, r_{\text{hi}}^{\text{dec}}] = [1.10, 2.19]$  pJ/op. This is narrower than the cross-format envelope of Table 1, which pins no format: we assume proof-of-learning and zero-knowledge proof-of-training protocols (Assumption 2.2) let the verifier confirm the declared work ran in its declared format, so the declared edges are the fp16 cells’ own;
- the overhead width, from Experiment 2:  $\Delta P_0 = 41.2$  W;
- the covert cost, from Experiment 3’s marginal lower limits:  $r_{\text{lo}}^{\text{cov}} = 0.51$  pJ/op for covert int8 ( $\gamma = 2$ ) or 0.76 pJ/op for covert fp16 ( $\gamma = 1$ );
- capacity  $C_{\text{max}}$  and  $\gamma$  (Equation (19)), from the two formats’ published peaks rather than the rates our kernels achieve;
- the declared utilisation  $h$ : we take the worst case  $\beta^* = \max_h \beta(h)$  (Equation (22)).

Together these give, with covert int8,

$$\beta^* = 1.16 \quad \text{at} \quad h^* = 0.42, \quad (34)$$

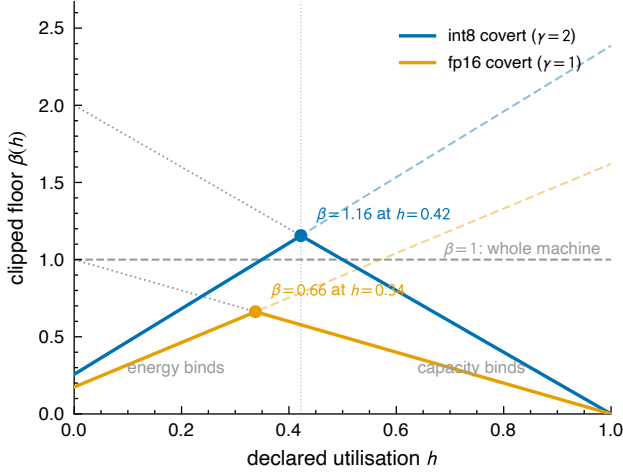


Figure 7. The capacity-clipped floor  $\beta(h)$  (Equation (21)) from the jointly feasible fp16-declared bands, for covert int8 ( $\gamma = 2$ , blue) and covert fp16 ( $\gamma = 1$ , orange), both priced at Experiment 3’s marginal lower limits. Faint dashed lines are the unclipped energy-only widths  $\beta_{\text{en}}(h)$  ( $\beta_{\text{en}}(1) = 2.39$  and  $1.62$ ): energy is the binding constraint left of each crossing, capacity right of it, and both bind at  $h^*$ . Dotted lines are the capacity clips  $(1 - h)\gamma$ ; markers are the worst cases of Equation (22).

and an energy-only width at full declared utilisation of  $\beta_{\text{en}}(1) = 2.39$ . With covert work held to fp16 the pair is  $\beta^* = 0.66$  at  $h^* = 0.34$  and  $\beta_{\text{en}}(1) = 1.62$ . Figure 7 traces  $\beta(h)$  for both scenarios.

Three remarks. First,  $\beta_{\text{en}}(1)$  is an energy-only diagnostic, not a physical floor: at full declared utilisation the capacity clip  $(1 - h)\gamma$  is zero, so no covert work fits and  $\beta(1) = 0$ . Second, published peaks give the covert stream more room than it can really use, so this is the adversary-favorable direction; Appendix D quantifies the difference on our measured peaks. Third, we can measure how much of this conceded width a real attack occupies. On the same card we ran pairs of runs matched in total energy: an honest run executing  $N$  declared fp16 iterations on expensive operands, and a cheating run executing the same iterations on cheap operands plus a covert int8 fill sized to draw the same energy. Every cheating run passes the no-alarm test of Equation (15) while executing 0.19–0.41 machines of covert work at  $h = 0.10$ –0.216. The identified set is therefore at least 0.41 wide at  $h = 0.216$ , against the model’s  $\beta(0.216) = 0.72$ ; the attack does not exhaust the bound because no single schedule reaches every edge of the band at once. Restricting the model to utilisations the attack can occupy ( $h \lesssim 0.24$ ) would lower the worst case to  $\approx 0.77$ , but that reflects our kernels rather than the adversary’s best, so we do not quote it as a bound. Appendix E records the protocol.

## 5. The capability ladder

The headline of Section 4.2,  $\beta^* = 1.16$  (Equation (34)), is a worst-case bound under Assumptions 2.1 and 2.2 alone: the verifier reads the power and checks the declared precision, and nothing else. In practice the verifier can make a series of reasonable checks and assumptions to remove ambiguity and push down  $\beta^*$ . Writing the energy-only width of Equation (18) with its inputs exposed,

$$\beta_{\text{en}}(h) = \beta_{\text{en}}(h; r_{\text{hi}}^{\text{dec}}, r_{\text{lo}}^{\text{dec}}, r_{\text{lo}}^{\text{cov}}, \Delta E_0), \quad (35)$$

we see that the triple of Equation (14) together with the overhead width are the only dials to turn to affect  $\beta_{\text{en}}$

*The declared edges  $r_{\text{hi}}^{\text{dec}}$  and  $r_{\text{lo}}^{\text{dec}}$ .* These are accessible to the verifier, at least in principle, because they price work the prover has declared and the verifier may re-run. The factorisation of Equation (25) maps the available actions one can take to narrow that width: the declared precision is checked, so  $w_{\kappa} = 1$ , while the remaining factors  $w_{\alpha} w_C w_V$  provide open degrees of freedom. Observations can remove the  $(w_V, w_C)$  freedom; re-executing the declared work on the verifier’s own hardware measures  $\bar{r}^{\text{dec}}$  directly, bypassing  $w_{\alpha}$  rather than pinning it.

*The covert price  $r_{\text{lo}}^{\text{cov}}$ .* By definition, this is not accessible to the verifier; the prover still chooses the covert format, operands and locality freely, and the meter never resolves them. However, a threat-model or an assumption about what the adversary is trying to do does introduce restrictions. For example, assuming that the covert work is useful, or that we know its numeric format.

*The overhead width  $\Delta E_0$ .* Similarly, this dial is not accessible to the verifier but can be restricted through reasonable assumptions. For example, assuming that the card is executing throughout the window being priced, so its overhead cannot be that of a cold card.

The rest of this section takes these restrictions in turn. They compound. Each one keeps everything granted above it and turns one more dial, so the bound can only fall as we descend: from the headline  $\beta^* = 1.16$ , where the meter and the declared format are all the verifier has, to  $\beta^* = 0.059$  at the bottom, with every restriction in place. Table 2 and Figure 8 walk down that ladder.

**Observe the clock and locality (unpriced; dial  $r_{\text{hi}}^{\text{dec}}$ ).** This rung would set  $w_V \rightarrow 1$  and  $w_C \rightarrow 1$ , leaving the declared band at its activity core  $w_{\alpha}$  alone. Both are hardware-wide observables the power meter does not measure. We deliberately quote no number for it, for two reasons.

First, no verifier can do this today. Three routes might get there: the card could attest its own clock telemetry, the facility could lock its clocks by policy, or an external

| verifier gains                                             | factor acted on, and the edges it leaves                          | $\beta^*$ | $\beta_{\text{en}}(1)$ |
|------------------------------------------------------------|-------------------------------------------------------------------|-----------|------------------------|
| precision declared & checked <sup>†</sup>                  | $w_\kappa = 1$ ; band [1.10, 2.19]                                | 1.16      | 2.4                    |
| clock & locality observed                                  | $w_V, w_C \rightarrow 1$ ( <i>unpriced</i> )                      | —         | —                      |
| declared work re-executed <sup>‡</sup>                     | <i>bypasses</i> $w_\alpha$ : band $\rightarrow$ gap               | 0.34      | 0.35                   |
| covert work on useful data <sup>‡</sup>                    | denominator: 0.51 $\rightarrow$ 2.53                              | 0.071     | 0.072                  |
| overhead band pinned to the window <sup>‡</sup>            | $\Delta P_0$ : 41.2 $\rightarrow$ 31.3 W                          | 0.059     | 0.059                  |
| <i>branch — covert format known to be fp16<sup>§</sup></i> |                                                                   |           |                        |
| at the precision rung                                      | $\gamma$ : 2 $\rightarrow$ 1; denominator 0.51 $\rightarrow$ 0.76 | 0.66      | 1.6                    |
| at the re-execution rung                                   | <i>as above</i> , band $\rightarrow$ gap                          | 0.23      | 0.24                   |

Table 2. The capability ladder. Each row grants the verifier one more capability, cumulatively from the top, and moves one dial of Equation (35); the top row is the headline of Equation (34). Every row is the clipped worst case  $\beta^* = \max_h \beta(h)$  (Equation (22)) for the fp16-declared, int8-covert case study of Section 4.2, computed directly from the edges beside it; rate edges are in pJ/op. We include  $\beta_{\text{en}}(1)$ , the energy-only width of Equation (18), as an additional diagnostic. The clock and locality row is left unpriced, for the two reasons given in Section 5. Disclosed conditions, inherited by every row below their first appearance: <sup>†</sup> — the declared work is executed in its declared format; <sup>‡</sup> — re-execution transfers at an observed operating point. <sup>§</sup> — the branch restricts covert work to fp16 instead of int8, and is quoted only at the two rungs where its denominator is measured: it acts on the same dial as the useful-data row, so the two do not compose (Section 5).

sensor could read the clock off the electromagnetic signature (Section 6). None of the three is currently deployable.

Second, and independently, we have not measured  $w_V$ . Neither Experiment 1 nor the operand sweep of Appendix G sweeps the clock. In both, the governor held it inside a  $\pm 3\%$  window. So the  $\approx 1.5$  we carry for  $w_V$  is Section 4.2’s device-physics estimate, not a measurement (Table 1).

**Re-execute the declared work** (1.16  $\rightarrow$  0.34, **conditional; dial  $r_{\text{hi}}^{\text{dec}}$** ). The verifier re-runs the declared work on its own hardware and measures what it costs. It reads the operands that actually ran, removing the prover’s freedom to claim cheap data.

This collapses the declared band  $r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}} \rightarrow \epsilon$ , where  $\epsilon$  is a small but non-zero residual that quantifies the difference between the verifier’s hardware and the prover’s hardware. The verifier’s trusted cluster is not the monitored facility, and energy per operation depends on the accelerator, its firmware, and its cooling. So the measured rate transfers only up to this hardware gap.

We measure the easiest version of the gap by re-measuring every band edge on three further A100s of the same SKU. The edges agree to 2–3% across devices (Appendix F). Same-SKU re-execution therefore transfers bands at the few-per-cent level. However, across accelerator generations, firmware and cooling the gap is likely larger, and we have not measured it here.

To move forwards we assume that  $\epsilon$  is a few per cent of the declared rate. We take  $\epsilon \approx 0.05$  pJ/op, informed by the same-SKU spread but not measured across hardware, which gives  $\beta^* = 0.34$  at  $h^* = 0.83$ . We treat this as a sensitivity analysis in  $\epsilon$  rather than a measured bound: at the same-SKU spread itself (0.02–0.04 pJ/op) it reads  $\beta^* = 0.29$ –0.32; at our stated 0.05 pJ/op,  $\beta^* = 0.34$ ; at a

cross-vendor 0.2 pJ/op,  $\beta^* \approx 0.54$ .

Moreover, even in the case where  $\epsilon = 0$ , re-execution pays only at an observed operating point, cf. the unpriced clock rung above. The verifier measures the declared work at its own clock. If the prover’s clock is unobserved the prover may run the declared work anywhere on the DVFS curve. The transfer error is then not the hardware gap but the whole DVFS factor,  $\sim 1.5$ . So a verifier with only the meter, declarations and re-execution, but no on-site sensing, stalls at  $\beta^* \approx 1$ . Re-executing the declared work buys it almost nothing that checking the precision had not already bought. Methods for pinning the clock are therefore a prerequisite for this rung, and no methods are currently deployable.

**Restrict covert work to useful data** (0.34  $\rightarrow$  0.071; **dial  $r_{\text{lo}}^{\text{cov}}$ ; Experiment 4**). This rung is granted by restrictions introduced by the threat model. Covert work is unseeable by definition: no declaration reaches it, so nothing but the silicon bounds its cost. That bound is the int8 marginal edge, 0.51 pJ/op. But the cheapest operations at that edge are operand zeros. In practice hidden zero-ops are no threat as they compute nothing. An adversary who uses the hidden slack to perform useful computation pays more than the silicon floor.

We measured this by taking an int8 quantised transformer block built from one trained Pythia-6.9B layer, run on real text activations rather than synthetic patterns, with the quantisation scales applied. Every metered operation is counted, both the int8 matmuls and the fp16 attention. We swept that block as the covert dose inside Experiment 3’s construction (Equation (31)), varying only the number of covert blocks between windows. We ran two shapes of the block, which differ in how much of the operation count is fp16 attention rather than int8 matmul, and so bound the per-operation cost from either side. Regressing window energy on covert nom-

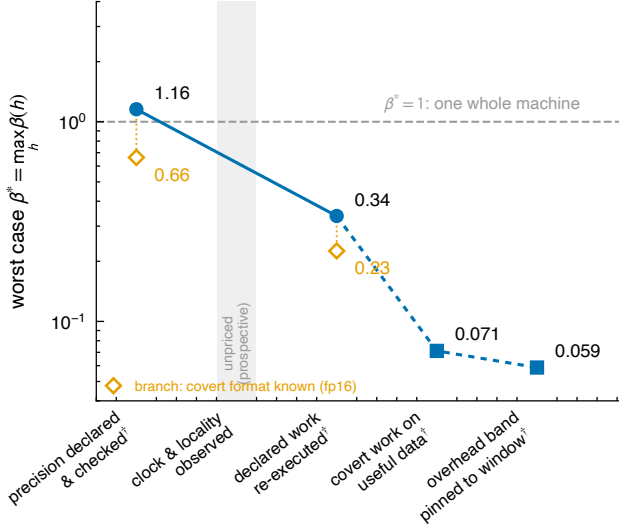


Figure 8. The capability ladder of Table 2 on a log scale; every plotted value is the clipped worst case  $\beta^* = \max_h \beta(h)$ . Filled circles are capabilities the verifier gains; square rungs are forced by the threat model rather than granted by a channel; the shaded slot is the unpriced prospective clock/locality rung, and daggered labels carry the disclosed conditions of Table 2. Hollow diamonds are the covert-format branch, dropped from the two rungs it re-prices: it is not a continuation of the chain, and does not compose with the rungs below. The  $\beta^* = 1$  line marks one whole machine of conceded ambiguity.

inal operations gives a marginal cost of 2.54 and 3.21 pJ/op for the two shapes, both fits linear to  $R^2 > 0.999$ . We carry the one-sided 95% lower limits,  $r_{\text{useful}} = [2.53, 3.16]$  pJ/op.

Restricting the adversary to work worth doing therefore raises the covert edge to the bottom of that band,  $0.51 \rightarrow 2.53$  pJ/op, i.e. every slack joule now buys the adversary  $\approx 4.9\times$  fewer useful operations than silicon-floor accounting would concede. The rung drops to  $\beta^* = 0.071$ .

**Pin the overhead band to the window being priced** ( $0.071 \rightarrow 0.059$ ; dial  $\Delta E_0$ ; Experiment 2). Like the useful-data rung, this rung is granted by restrictions introduced by the threat model. The band of Equation (29) spans Experiment 2’s twenty-five idle windows. Its low edge, 62.9 W, is the cold-card cell — the device before any CUDA context exists. But in practice the card is not cold. The declared burst runs for  $hT$  and covert work fills the rest of the window. The card is therefore executing from one end of the window to the other, and its operands stay in memory throughout. A card in that state cannot draw a cold card’s overhead. Restricting the band to the fifteen windows in which a context and operand pool were resident lifts the low edge to 73.4 W. The high edge does not move: the 96.2 W peak is the post-heavy-kernel transient, and the priced window’s declared burst is a heavy kernel. With the

same  $\pm 5\%$  meter tolerance,

$$P_0 \in [69.7, 101.0] \text{ W}, \quad \Delta P_0 = 31.3 \text{ W}, \quad (36)$$

which is 0.76 of the published width, so the rung falls from 0.071 to 0.059. The restriction is not specific to the bottom of the ladder: at the re-execution rung it reads 0.28 in place of 0.34, and at the headline 1.12 in place of 1.16, where the declared band and not the overhead carries the width.

Even after pinning, most of the bottom rung is overhead. The overhead term  $b = \Delta P_0 / (r_{\text{lo}}^{\text{cov}} F_{\text{max}}^{\text{dec}})$  is about 70% of  $\beta^*$  at the bottom rung (0.040 of 0.059) and at the re-execution rung (0.195 of 0.28). Reporting  $P_0$  directly, over a trusted on-site channel rather than bracketing it, would leave only the meter tolerance,  $\Delta P_0 \approx 8.5 \text{ W}$ , and a bottom rung of  $\beta^* = 0.030$ .

**Branch: know the covert format** ( $1.16 \rightarrow 0.66$ ; dials  $r_{\text{lo}}^{\text{cov}}$  and  $\gamma$ ). This restriction is enabled by the threat model. We assume that the covert numeric format is known. This is a reasonable assumption: if compute capacity is being stolen to train a frontier model, that training is highly likely to run in fp16 or bf16 with fp32 accumulation. int8 pre-training is not a viable route to the capability an adversary wants.

Restricting covert work to fp16 moves two things. First, the covert denominator rises from the int8 marginal edge to the fp16 one,  $0.51 \rightarrow 0.76$  pJ/op, both measured in Experiment 3 (Section 4.2). That is a factor 1.47 on the declared and the overhead term alike, since both carry  $r_{\text{lo}}^{\text{cov}}$  in their denominator. Second, the capacity ratio falls,  $\gamma = 2 \rightarrow 1$ : covert operations no longer retire at twice the declared rate, so the clip becomes  $1 - h$ . At the headline rung the two together give  $\beta^* = 0.66$ . Carried down to the re-execution rung the branch reads 0.23 in place of 0.34. Figure 7 traces both scenarios.

## 6. Discussion

We have shown how a power meter can restrict hidden compute to an interval,  $\beta$ . For a meter that also checks the declared precision, the worst case is  $\beta^* = 1.16$  machines of covert compute. A set of reasonable assumptions made by the verifier push  $\beta^*$  down, to a lowest value of  $\beta^* = 0.059$ , with every additional assumption of Section 5 in place.

**Limitations.** The above claim on the value of  $\beta$  has a series of caveats as follows. (i) The empirical values of Section 5 come from one vendor, one GPU model, one driver, and one cluster. Consequently the quoted values of  $\beta$  are A100-specific, though the framework and closed form of Section 3 is general. (ii) The covert denominator behind the headline,  $r_{\text{lo}}^{\text{cov}} = 0.51$  pJ/op, is measured, but on synthetic operands: its corner cell is int8 on zeros, which compute nothing. Experiment 4 prices genuinely useful covert work

instead, and the bottom rungs rest on the lower edge of that measurement. (iii) The hardware gap behind the re-execution rung ( $\approx 0.05$  pJ/op) is an assumption. The same-SKU piece is grounded - band edges agree to 2–3% across three A100s (Section 4.2). However the cross-hardware gap (different accelerators, firmware, cooling) remains unmeasured. (iv) The platform denied clock-locking and so DVFS was never swept. Accordingly the voltage factor  $w_V \approx 1.5$  of Equation (25) rides on literature numbers rather than our own. The declared band was therefore sampled with the governor holding the clock inside a  $\pm 3\%$  window, whereas the threat model gives the prover the clock: a prover free to slide DVFS reaches a wider declared band and possibly a cheaper covert floor, raising both  $s$  and  $b$  in Equation (18). (v) Energy was read from on-die telemetry, whereas in practice the verifier reads a retrofitted meter on the power feed. The on-die counter carries no published GA100 accuracy specification, so the  $\pm 5\%$  tolerance we apply is adopted from NVML’s documented Fermi/Kepler figure, and the  $\sim 100$  ms telemetry cadence is an empirical characterisation (Yang et al., 2023), not a vendor specification. That retrofitted meter carries its own calibration band, and it also sees conversion and cooling draw the die counter never reports. Both widen  $\Delta P_0$ , and so widen  $\beta$ . (vi) We measured one device, whereas a facility is many devices behind shared power delivery and cooling. We do not offer a facility-level, multi-device version of Equation (18). Each device carries its own capacity clip and its own utilisation. The prover chooses how the declared work is spread across devices and formats, so the width of the identified set depends on an allocation the prover is free to pick. How to make the multi-device floor precise is an open question. (vii) The floor is a worst case in a strong sense. It prices a window in which the declared work is at its cheapest, the overhead at its lowest, and the covert work at its cheapest — all three at the same time. We do not show that any single schedule can do all three at once. If none can, the true ambiguity is narrower than the  $\beta$  we quote (Equation (41)), and we have conceded the adversary more than the silicon allows.

**Richer telemetry** Throughout this work we have dealt exclusively with time-integrated power, i.e. energy. Because  $r(t)$  and  $P_0(t)$  may vary arbitrarily within their bands, the time-resolved power trace carries no information about total compute beyond the integral. However real hardware is not perfectly free and does not vary arbitrarily. Instead power-ramp limits, DVFS governor dynamics, the instantaneous clip  $F(t) \leq F_{\max}$ , and any declared schedule the prover commits to all constrain how the trace can move. A verifier who models these constraints can likely extract more from the resolved trace than from its integral. The time-resolved trace also carries different information such as workload cadence or training-step periodicity. This could be useful for attacking different questions about what kind of compute

ran (e.g. training vs. inference) rather than how much; that question needs different machinery and we do not treat it here.

Moreover, we have focused exclusively on a single analogue measurement channel: power. A verifier with richer instrumentation or multiple simultaneous channels (e.g. per-rail power, electromagnetic, thermal), cryptographically authenticated telemetry (the proof-of-learning and zero-knowledge proof-of-training protocols of Assumption 2.2), or independently controlled measurements the prover cannot shape (locked clocks, tamper-resistant attested counters) would likely be able to further constrain the total covert compute. For example, take the clock. The power meter cannot see it, which is why Section 5 leaves the clock-and-locality rung unpriced. An antenna beside the rack might see it instead. A circuit switching at frequency  $f$  radiates at  $f$  and its harmonics, and magnetic emanations from a GPU’s power delivery are measurable in practice (Maia et al., 2022). A verifier who could read the clock this way would get the voltage too, since that follows from the hardware’s  $V$ – $f$  table. The DVFS factor  $w_V$  would fall to 1, and the declared band would narrow to its activity core  $w_\alpha$  alone. We defer the study of multiple measurement channels to a future work.

## 7. Conclusion

So, can power draw constrain covert compute? Not on its own. A meter that reads the energy of a window, and knows the numeric format the declared work ran in, leaves room for more undeclared computation than the machine’s whole declared capacity:  $\beta^* = 1.16$ . But the meter is rarely all a verifier has. Re-running the declared work on trusted hardware, restricting hidden work to useful computation, and pinning the idle draw to a card known to be busy bring  $\beta^*$  to 0.059, six per cent of one machine. What separates the two numbers is information available to the verifier, and the closed form prices each piece of it.

That is the case for reading power alongside other channels rather than by itself. The clock is the clearest example: the meter cannot see it, an antenna beside the rack might, and a verifier who reads it fixes the voltage too and narrows the declared band to its activity core. Pricing that rung, replacing the on-die counter with a retrofitted meter on the power feed, and carrying the floor from one device to a whole facility are natural next steps.

## References

- Ansari, S. Hardware-level governance of AI compute: A feasibility taxonomy for regulatory compliance and treaty verification, April 2026. URL <https://arxiv.org/abs/2604.04712>.

- Arefin, S. E. and Serwadda, A. Exploiting HDMI and USB ports for GPU side-channel insights, October 2024. URL <https://arxiv.org/abs/2410.02539>. Published at ACM CODASPY 2025.
- Baker, M., Kulp, G., Marks, O., Brundage, M., and Heim, L. Verifying international agreements on AI: Six layers of verification for rules on large-scale AI development and deployment, July 2025. URL <https://arxiv.org/abs/2507.15916>.
- Bengio, Y., Clare, S., Prunkl, C., Andriushchenko, M., Bucknall, B., et al. International AI safety report 2026, February 2026. URL <https://arxiv.org/abs/2602.21012>.
- Chandrakasan, A. P., Sheng, S., and Brodersen, R. W. Low-power CMOS digital design. *IEEE Journal of Solid-State Circuits*, 27(4):473–484, 1992. doi: 10.1109/4.126534.
- Chatterjee, N., O’Connor, M., Lee, D., Johnson, D. R., Keckler, S. W., Rhu, M., and Dally, W. J. Architecting an energy-efficient DRAM system for GPUs. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 73–84, 2017. doi: 10.1109/HPCA.2017.58.
- Chaudhuri, A., Shukla, S., Bhattacharya, S., and Mukhopadhyay, D. “Energon”: Unveiling transformers from GPU power and thermal side-channels, August 2025. URL <https://arxiv.org/abs/2508.01768>.
- Choukse, E., Warriar, B., Heath, S., Belmont, L., Zhao, A., Khan, H. A., Harry, B., Kappel, M., Hewett, R. J., Datta, K., et al. Power stabilization for AI training datacenters, August 2025. URL <https://arxiv.org/abs/2508.14318>. Microsoft.
- Clark, S. S., Ransford, B., Rahmati, A., Guineau, S., Sorber, J., Xu, W., and Fu, K. WattsUpDoc: Power side channels to nonintrusively discover untargeted malware on embedded medical devices. In *2013 USENIX Workshop on Health Information Technologies (HealthTech 13)*. USENIX Association, 2013.
- Gangwal, A., Piazzetta, S. G., Lain, G., and Conti, M. Detecting covert cryptomining using HPC, 2019. URL <https://arxiv.org/abs/1909.00268>. Published at CANS 2020.
- Gao, Y., Qiu, H., Zhang, Z., Wang, B., Ma, H., Abuadba, A., Xue, M., Fu, A., and Nepal, S. DeepTheft: Stealing DNN model architectures through power side channel, September 2023. URL <https://arxiv.org/abs/2309.11894>. Published at 2024 IEEE Symposium on Security and Privacy (S&P).
- Garg, S., Goel, A., Jha, S., Mahloujifar, S., Mahmood, M., Policharla, G.-V., and Wang, M. Experimenting with zero-knowledge proofs of training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023. doi: 10.1145/3576915.3623202.
- Heim, L. and Koessler, L. Training compute thresholds: Features and functions in AI regulation, 2024. URL <https://arxiv.org/abs/2405.10799>.
- Horowitz, M. 1.1 computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers*, pp. 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323.
- Hu, X., Liang, L., Li, S., Deng, L., Zuo, P., Ji, Y., Xie, X., Ding, Y., Liu, C., Sherwood, T., and Xie, Y. DeepSniffer: A DNN model extraction framework based on learning architectural hints. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 385–399. ACM, 2020.
- Jia, H., Yaghini, M., Choquette-Choo, C. A., Dullerud, N., Thudi, A., Chandrasekaran, V., and Papernot, N. Proof-of-learning: Definitions and practice. In *2021 IEEE Symposium on Security and Privacy (SP)*, 2021. doi: 10.1109/SP40001.2021.00106.
- Latif, I., Newkirk, A. C., Carbone, M. R., Munir, A., Lin, Y., Koomey, J., Yu, X., and Dong, Z. Single-node power demand during AI training: Measurements on an 8-GPU NVIDIA H100 system. Technical report, Office of Scientific and Technical Information (OSTI), 2025. URL <https://www.osti.gov/biblio/2555926>.
- Maia, H. T., Xiao, C., Li, D., Grinspun, E., and Zheng, C. Can one hear the shape of a neural network? Snooping the GPU via magnetic side channel. In *31st USENIX Security Symposium (USENIX Security 22)*, pp. 4383–4400. USENIX Association, 2022.
- Monfared, S. K., Ganji, F., Holcomb, D., and Tajik, S. Timing and memory telemetry on GPUs for AI governance, 2026. URL <https://arxiv.org/abs/2602.09369>.
- Naghibijouybari, H., Neupane, A., Qian, Z., and Abughazaleh, N. Rendered insecure: GPU side channel attacks are practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2139–2153. ACM, 2018.
- NVIDIA Corporation. NVIDIA A100 tensor core GPU architecture. Whitepaper, 2020. URL <https://www.nvidia.com/content/>

- dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf. Dense tensor-core peaks: 19.5 TFLOP/s fp32, 156 TFLOP/s tf32, 312 TFLOP/s fp16/bf16, 624 TOPS int8.
- NVIDIA Corporation. NVIDIA management library (NVML) API reference. Online documentation, 2026. URL <https://docs.nvidia.com/deploy/nvml-api/>. Device queries `nvmlDeviceGetPowerUsage` and `nvmlDeviceGetTotalEnergyConsumption`. The stated +/- 5% accuracy is given for Fermi and Kepler power readings; no accuracy specification is given for the total-energy counter on GA100.
- O’Gara, A., Kulp, G., Hodgkins, W., Petrie, J., Immler, V., Aysu, A., Basu, K., Bhasin, S., Picek, S., and Srivastava, A. Hardware-enabled mechanisms for verifying responsible AI development. In *ICML 2025 Workshop on Technical AI Governance (TAIG)*, 2025. URL <https://arxiv.org/abs/2505.03742>.
- Pallipadi, V. and Starikovskiy, A. The ondemand governor: Past, present, and future. In *Proceedings of the Linux Symposium*, volume 2, pp. 223–238, 2006.
- Patel, P., Choukse, E., Zhang, C., Goiri, Í., Warriier, B., Mahalingam, N., and Bianchini, R. POLCA: Power oversubscription in LLM cloud providers, 2023. URL <https://arxiv.org/abs/2308.12908>.
- Rahman, R. and Tajdari, S. Detecting hidden ML training with zero-overhead telemetry, 2026. URL <https://arxiv.org/abs/2606.19262>.
- Reuel, A., Bucknall, B., Casper, S., Fist, T., Soder, L., Aarne, O., Hammond, L., Ibrahim, L., Chan, A., Wills, P., Anderljung, M., Garfinkel, B., Heim, L., Trask, A., Mukobi, G., Schaeffer, R., Baker, M., Hooker, S., Solaiman, I., Luccioni, A. S., Rajkumar, N., Moës, N., Ladish, J., Bau, D., Bricman, P., Guha, N., Newman, J., Bengio, Y., South, T., Pentland, A., Koyejo, S., Kochenderfer, M. J., and Trager, R. Open problems in technical AI governance, 2025. URL <https://arxiv.org/abs/2407.14981>.
- Sastry, G., Heim, L., Belfield, H., Anderljung, M., Brundage, M., Hazell, J., O’Keefe, C., Hadfield, G. K., Ngo, R., Pilz, K., Gor, G., Bluemke, E., Shoker, S., Egan, J., Trager, R. F., Avin, S., Weller, A., Bengio, Y., and Coyle, D. Computing power and the governance of artificial intelligence, February 2024. URL <https://arxiv.org/abs/2402.08797>.
- Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., and Villalobos, P. Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2022. doi: 10.1109/IJCNN55064.2022.9891914.
- Shavit, Y. What does it take to catch a Chinchilla? Verifying rules on large-scale neural network training via compute monitoring, 2023. URL <https://arxiv.org/abs/2303.11341>.
- Tang, B. J., Chen, Q., Weiss, M. L., Frey, N., McDonald, J., et al. The MIT Supercloud workload classification challenge, 2022. URL <https://arxiv.org/abs/2204.05839>.
- The White House. Executive order 14110: Safe, secure, and trustworthy development and use of artificial intelligence. Federal Register 88 FR 75191, 2023. URL <https://www.federalregister.gov/documents/2023/11/01/2023-24283/>.
- The White House. Executive order 14148: Initial rescissions of harmful executive orders and actions. Federal Register Doc. 2025-01901, 2025. URL <https://www.federalregister.gov/documents/2025/01/28/2025-01901/>.
- Wei, J., Zhang, Y., Zhou, Z., Li, Z., and Al Faruque, M. A. Leaky DNN: Stealing deep-learning model secret with GPU context-switching side-channel. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 125–137. IEEE, 2020.
- Xu, H., Gong, C., Liu, B., Zheng, H., Chen, B., and Li, M. Wave: Leveraging architecture observation for privacy-preserving model oversight. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLoS)*, 2026. doi: 10.1145/3779212.3790247.
- Yang, Z., Adámek, K., and Armour, W. Part-time power measurements: nvidia-smi’s lack of attention, 2023. URL <https://arxiv.org/abs/2312.02741>.

## A. From the CMOS power law to the exchange rate

Section 2.1 sketches the route from the CMOS power law to the exchange rate; the factor decomposition of Section 4.1 additionally adopts the factored model Equation (23). We give the derivation in full here and locate the terms the factorisation drops. Resolving the static term of Equation (1), total chip power separates into a dynamic (switching) part and a static part,

$$P = P_{\text{dyn}} + P_{\text{stat}}, \quad P_{\text{dyn}} = \alpha C V^2 f, \quad P_{\text{stat}} = V I_{\text{leak}} + P_{\text{fixed}}, \quad (37)$$

with  $\alpha$  the activity factor,  $C$  the total capacitance of the logic (so  $\alpha C$  is the capacitance switched per cycle),  $V$  the supply voltage,  $f$  the clock,  $V I_{\text{leak}}$  the leakage power, and  $P_{\text{fixed}}$  the cooling, I/O, and idle draw. The forward model Equation (2),  $P = rF + P_0$ , is slope–intercept in the operation rate  $F$ , so the exchange rate is the *marginal* energy per operation and the overhead is the intercept,

$$r = \frac{\partial P}{\partial F}, \quad P_0 = P|_{F=0}. \quad (38)$$

Experiment 2 measures the intercept of Equation (38) directly from idle cells (Section 4.2) and checks the slope against a sweep of sustained nominal-op rate (Appendix C).

**The dynamic term gives  $r$ .** The operation rate is  $F = N_{\text{op}} f$ , where  $N_{\text{op}}$  is the number of nominal operations the datapath retires per cycle at the operating format (an idle unit lowers the activity  $\alpha$ , not the clock  $f$ ). At a fixed format the clock is thus the knob through which the operation rate moves,  $\partial f / \partial F = 1 / N_{\text{op}}$ , and the marginal rate of Equation (38) follows by the chain rule, holding  $\alpha$ ,  $C$ , and  $V$  fixed,

$$r = \frac{\partial P_{\text{dyn}}}{\partial F} = \frac{\partial P_{\text{dyn}}}{\partial f} \frac{\partial f}{\partial F} = \frac{\alpha C V^2}{N_{\text{op}}} = \alpha \left( \frac{C}{N_{\text{op}}} \right) V^2, \quad (39)$$

and the clock drops out: energy per operation does not depend on how fast the operations run, only on what each switching event costs and how many operations it delivers. The rate is constant in  $F$ , so  $P_{\text{dyn}} = rF$  is linear through the origin, exactly the affine form the forward model Equation (2) assumes and Experiment 2’s rate ladder checks (Appendix C). The capacitance switched per operation,  $C / N_{\text{op}}$ , has two structural sources: the arithmetic datapath, whose width is set by the numeric format, and the data-movement path, set by where the operands reside and which kernel runs. Factoring it accordingly,

$$\frac{C}{N_{\text{op}}} = \kappa(\text{precision}) C_{\text{eff}}(\text{locality, kernel}), \quad (40)$$

with  $\kappa$  collecting the format scale (datapath width and the operation packing  $N_{\text{op}}$ ) and  $C_{\text{eff}}$  the locality and kernel structure, and substituting into Equation (39), gives the model Equation (23). This is a re-parameterisation by physical origin, not a unique factorisation:  $\kappa$ ,  $\alpha$ , and  $C_{\text{eff}}$  are all switching quantities, grouped by cause—format, operand data, memory locality—so that each maps to a single verifier category.

**Short-circuit sits in  $r$ , inside the switched capacitance.** Chandrakasan et al. (1992) also carry a *short-circuit* term for the direct-path current that flows while an input transitions and the NMOS and PMOS networks briefly conduct together. It is triggered by the same switching events as  $P_{\text{dyn}}$ , so it scales with the operation rate and survives the marginal derivative of Equation (38). In a well-designed circuit it is a small fraction of the switching power, so we fold it into the effective switched capacitance  $C_{\text{eff}}$  of Equation (40): it lands in  $r$ , but introduces no new verifier category.

**Leakage sits in  $P_0$ , not  $r$ .** The static power  $P_{\text{stat}}$  of Equation (37) flows whether or not the machine computes, so  $\partial P_{\text{stat}} / \partial F = 0$ : by Equation (38) it contributes nothing to the marginal rate  $r$  and is entirely the intercept  $P_0$ , where the derivation of Equation (10) already places it and carries it as a bounded nuisance band. A leakage-per-operation term  $r_{\text{leak}} = P_{\text{stat}} / F$  would appear only if  $r$  were defined as the *average* energy per operation,  $P / F$ , which mixes in the intercept and is operating-point dependent; we keep the marginal definition Equation (38), consistent with Equation (2), and so carry no separate leakage term in  $r$ .

## B. What kind of bound the floor is

The maximisation that produced Equation (21) was taken over a *box*: each of the three bands ranges freely, so the prover may sit at  $r_{\text{lo}}^{\text{dec}}$ ,  $E_{0,\text{lo}}$  and  $r_{\text{lo}}^{\text{cov}}$  all at once. That combination is the *cheating corner*, and nothing in the derivation guarantees a single schedule reaches it.

**Assumption B.1** (Joint feasibility of band corners). There exist admissible workloads attaining  $(\bar{r}^{\text{dec}}, E_0, \bar{r}^{\text{cov}}) = (r_{\text{lo}}^{\text{dec}}, E_{0,\text{lo}}, r_{\text{lo}}^{\text{cov}})$  jointly and under shared constraints: the *same* observation window  $T$ , the *same* declared work  $C_{\text{dec}}$ , and one device in a *common thermal and clock state* whose resource-time the two streams multiplex at their *achieved* throughputs.

The assumption is not free, and the overhead band shows why. Our  $E_{0,\text{lo}}$  is measured on an idle card (Section 4.2), whereas the window Equation (21) prices is running a declared burst and a covert fill back to back; a window like that does not draw an idle card’s overhead. The bands are also coupled through the clock and the thermal state, which the box treats as independent. So the corner is an outer relaxation of what a device can schedule, and

$$\text{true identified-set width} \leq \beta(h), \quad (41)$$

with equality only under Assumption B.1. The direction is the safe one for a security claim—we concede the adversary more than the silicon may allow—but  $\beta$  is a worst case in the strong sense, and the gap in Equation (41) is unquantified by the formula alone. Closing it from below takes a constructed schedule rather than an algebraic bound, which is what Appendix E does.

One further piece of bookkeeping keeps the box honest. Because both the energy per operation and the capacity are format-dependent, we evaluate the floor inside a single jointly feasible scenario—one declared format and one covert format sharing one window, with the declared band,  $C_{\text{max}}$  and  $h$  all referring to the declared format, and the covert cost and covert peak to the covert format. Combining extrema that no single schedule attains, such as pricing one format’s most expensive cells against another’s capacity, is excluded by construction. The prover may switch formats between the two streams at no cost, which is again the adversary-favorable direction.

## C. What the bound requires of the measurements

Appendix A derives the affine forward map Equation (2) from device physics. The security bound does not need it. This appendix states the weaker condition it does need, and why the two band edges are estimated by different means.

Two roles, two estimands. The declared edges enter Equation (17) only through their *difference*  $r_{\text{hi}}^{\text{dec}} - r_{\text{lo}}^{\text{dec}}$  at a common saturated rate, so any baseline amortised equally into both edges cancels and a measured total quotient  $E/C$  is the correct estimand for the numerator (the overhead enters once, separately, as  $\Delta E_0$ ). The covert cost enters as the *denominator*  $r_{\text{lo}}^{\text{cov}}$ , where nothing cancels: the bound is a valid security guarantee only if  $r_{\text{lo}}^{\text{cov}}$  is a genuine one-sided *lower* bound on the *incremental* covert energy. Write the true draw as  $P = g_x(F_{\text{dec}}, F_{\text{cov}})$ , some device-dependent function of what the silicon executes, with the subscript  $x$  standing for the execution details the verifier never sees. The condition is then

$$g_x(F_{\text{dec}}, F_{\text{cov}}) - g_x(F_{\text{dec}}, 0) \geq r_{\text{lo}}^{\text{cov}} F_{\text{cov}}, \quad (42)$$

for every admissible execution  $g_x$ . Under only this incremental condition the honest energy slack of Equation (17) still divides by  $r_{\text{lo}}^{\text{cov}}$ , so the floor holds without the global affine map of Equation (2); the affine model is the special case  $g_x = P_0 + r_{\text{lo}}^{\text{dec}} F_{\text{dec}} + r_{\text{lo}}^{\text{cov}} F_{\text{cov}}$ , which additionally makes  $\beta_{\text{en}}(h)$  linear in  $h$ . We therefore identify  $r_{\text{lo}}^{\text{cov}}$  by a dose–response intervention (Experiment 3 for the generic int8 edge, Experiment 4 for the useful-operand edge), not by fitting a power–rate curve; a total-quotient covert edge is inadmissible here, and every covert edge the bound uses is a marginal dose slope.

**The rate ladder, and why its slope is not that estimand.** Alongside the idle cells that give the overhead band, Experiment 2 sweeps a *rate ladder*: ten fp16 matmul shapes spanning the machine’s sustained throughput—three small cubes ( $256^3$  to  $1024^3$ ) that are launch-bound, two skinny- $K$  shapes ( $8192 \times 8192 \times 64$  and  $\times 256$ ) that are bandwidth-bound, and five large cubes ( $2048^3$  to  $8192^3$ ) that are compute-bound, a 60–90× spread in sustained operation rate at a single numeric format. Each window runs 100% busy at its shape’s own natural rate. We deliberately do *not* vary  $F$  by duty-cycling one kernel against idle: averaging a fixed busy power against a fixed idle power is linear in the duty fraction by construction, so that design would manufacture the affine form rather than test it. Operation counts are analytic ( $C = 2MNK$  per matmul, Section 2.1), so the abscissa is an exact op count divided by the measured wall time of the window and the ordinate is metered mean power, with the five idle variants as  $F = 0$  anchors ( $n = 65$  windows over 15 cells in all).

The resulting regression is a consistency diagnostic for the affine map, never an input to the bound, and the two roles above say why. The ladder’s shapes move  $C_{\text{eff}}$  and  $\alpha$  along with  $F$ , so the resulting spread in  $r$  is the same physical variation

Experiment 1 maps deliberately as the exchange-rate band—swept on purpose there, drifting as a side effect here. A slope fitted through them therefore averages over shapes whose non-covert factors vary together, where Equation (42) needs the derivative at *fixed* declared work. That is why the covert edge is identified by a dose–response intervention at a fixed declared burst rather than read off a power–rate curve. It is also why a pooled fit tests two claims at once—that power is affine in the rate, and that  $r$  is common across the shapes swept—of which the model itself denies the second. Read that way the ladder behaves as expected: restricted to the idle anchors and the five compute-bound cubes, like compared with like, the fit gives 1.65 pJ/op at  $R^2 = 0.99$ ; opened to all ten shapes it gives 1.60 pJ/op at  $R^2 = 0.852$ . The slope moves by 4% while the scatter grows sharply, so the launch- and bandwidth-bound shapes account for the residual and not for the rate. The like-for-like slope also sits inside the fp16 span [1.10, 2.19] pJ/op that Experiment 1 measures by total quotient in a separate sweep, which is the agreement two independent estimates of a common-format rate should show.

**Three kinds of quantity, three kinds of status.** The inputs to the bound do not share one statistical status, and we state each rather than claim a single coverage guarantee over all of them. The covert denominator—the one quantity a lower bound is genuinely sensitive to—is a one-sided 95% lower confidence limit on a regression slope (Experiments 3 and 4, and the useful-band dose slope), coverage-controlled by construction. That coverage is statistical, not physical. It bounds sampling error in the fitted slope for the workload families we measured, on this device and driver state; it does not certify that no admissible execution—an unseen kernel, schedule, operand encoding, or thermal state—sits below it. Every rung that divides by  $r_{lo}^{cov}$  is therefore conditional on the measured covert families bracketing the adversary’s cheapest edge, and the same reading applies to Experiment 4’s useful band, estimated from one trained transformer layer at two shapes. The declared band  $[r_{lo}^{dec}, r_{hi}^{dec}]$  enters only as a *difference* of per-cell-median edges at a common saturated rate, so the amortised baseline cancels and it acts as a band *width* rather than a point estimate needing its own coverage; its target population is the achievable per-shape operating points of the declared format on the audited A100 SKU, and the edges are that observed operating envelope. Used as design-time security edges they remain sampled operating points, not certified physical extrema. The overhead width  $\Delta P_0$  and the capacity and  $\gamma$  inputs are conservative worst-case *design* parameters, not sampled estimates at all:  $\Delta P_0$  is the envelope of the extreme observed idle windows, stretched outward by the adopted  $\pm 5\%$  meter tolerance (widening it only widens  $\beta$ ), and it is taken over windows rather than per-variant medians for that reason—an envelope wants its outermost observation, where an estimate would want a central one.  $C_{max}$  and  $\gamma$  are published hardware peaks. Nothing here claims simultaneous coverage over the rectangle these edges span; that rectangle is a deliberately conservative over-approximation, since no single schedule attains its combined extrema (Assumption B.1).

## D. Conservative accounting choices, quantified

Three accounting choices in the model bound are deliberately adversary-favorable; this appendix quantifies what each concedes.

**Published peaks vs. achieved rates.** The capacity clip Equation (20) is evaluated at the *published peak* capacities rather than the rates our kernels achieve. Peaks give the covert stream more room than it can really use, so the conceded  $\beta$  is larger than an achieved-rate evaluation would give. On our own measured peaks (246.9 Top/s fp16 and 307.7 Top/s int8, so  $\gamma = 1.25$  rather than 2) the bare-meter rung falls from 1.95 to 1.22 and the precision rung from 1.16 to 0.91. Assumption B.1 asks for achieved throughputs when it certifies that a particular *witness* is realisable in one window, and the matched-energy witnesses are checked that way (Appendix E); the model bound itself is deliberately evaluated at peaks.

**Marginal vs. total-quotient pricing of useful covert work.** The useful-data rung of Section 5 prices covert work at the *marginal* cost of an added covert block, one-sided 95% lower limits [2.53, 3.16] pJ/op. Pricing the same blocks as total quotients—window energy divided by covert operations, with baseline power amortised in—gives [3.18, 4.22] pJ/op instead. The marginal figure is the cheaper of the two, and a cheaper covert denominator concedes the adversary more operations per slack joule, so carrying it widens the rung. It is also the estimand the algebra asks for: the denominator of Equation (18) prices work added on top of a window whose overhead is already charged in its own term (Appendix C).

**Baseline power charged twice on the bare-meter rung.** Every entry of the bare-meter triple carries the estimand its algebraic role requires: the declared pair  $(r_{hi}^{dec}, r_{lo}^{dec}) = (19.5, 0.786)$  pJ/op as total quotients entering through their difference, the covert entry a one-sided marginal lower bound (0.515 pJ/op). This is why  $r_{lo}^{dec}$  and  $r_{lo}^{cov}$  stay distinct even here, where they describe the same physical corner: they are a third apart precisely because a total quotient amortises baseline power—leakage, cooling, idle draw—that a marginal cost does not, so a single number could satisfy at most

one of the two roles, and getting the denominator’s role wrong understates  $\beta$  (Appendix C). Keeping them distinct costs little in the numerator, where the ceiling dominates the difference, but it matters in the denominator: the overhead term  $\Delta P_0 / (r_{lo}^{cov} F_{max}) \approx 0.26$  is half again the 0.168 a total-quotient covert edge would give.

One conservatism does survive the bookkeeping. Baseline power is charged twice: once amortised inside the total-quotient ceiling  $r_{hi}^{dec}$ , and again as the overhead band  $\Delta E_0$  in its own term. Both charges widen the rung, so the figure we quote is an outer sensitivity rather than an exact width—the safe direction for a worst case, and small next to the rung’s dominant term.

## E. How much of the bound is real? The matched-energy witness protocol

Every number the body quotes is a maximum over the band box of Assumption B.1, and therefore an upper bound on the identified-set width rather than the width itself (Equation (41)). It is also a design-time statement: it bounds what *any* accepted window could hide, not the ambiguity of the particular window in front of you. This appendix asks how much of that headroom a real schedule can occupy. The instrument is the matched-pair witness of Section 4.2, which holds the trusted declaration and the observation window fixed by construction: run A executes  $N$  declared fp16 iterations on expensive operands (the  $r_{hi}^{dec}$  edge) and idles to a fixed  $T = 10$  s window; run B executes the *same*  $N$  iterations on cheap operands (the  $r_{lo}^{dec}$  edge) plus a covert int8 fill tuned to match A’s energy, padded to the same window. The witnesses reach  $h = 0.10, 0.15$  and  $0.216$ , where the accepted covert load reaches  $0.19, 0.29$  and  $0.41$  machines of declared-format capacity respectively — measured, capacity- and time-feasible lower bounds on the ambiguity.

**The energy match, and its margin.** Over eight interleaved repeats at  $h = 0.10$ , run A drew  $1291.3 \pm 4.4$  J and run B  $1279.3 \pm 5.6$  J, matching to  $|\Delta E|/E = 0.9\%$  while B hid  $3.5 \times 10^4$  covert int8 iterations behind the one-sided  $3\sigma$  no-alarm rule. Two features of the setup are conservative and one is thin. Clock locking is denied on the platform (Section 4.2), so the arms are matched by cadence rather than by a held clock: both reach the same 1410 MHz compute clock, the per-window *median* differing only through the idle-pad share, as in Experiment 3. Run A’s expensive burst draws  $\approx 500$  W against run B’s  $\approx 245$  W and so sits at the card’s software power cap, which truncates A’s measured honest energy; since the threshold is set from A’s mean and the covert dose is tuned against it, the cap tightens the bar B must clear. The thin part is the topmost witness: at  $h = 0.216$  the cheating arm’s spread is  $2.6\times$  the honest arm’s, and its most expensive repeat (1969.5 J) lands above the honest arm’s most expensive (1964.6 J), clearing the threshold by  $0.86\%$  of the honest mean. The  $0.41$  figure is the largest load we could get accepted, not a comfortable one.

**Achieved rates: lower witnesses, not corner attainment.** The achieved incremental rates are  $1.84$  pJ/op for the expensive declared arm (below the band ceiling  $2.19$ ),  $0.82$  for the cheap declared arm, and  $0.54$  for the covert fill (above the marginal edge  $0.51$ ): the witnesses meet the shared constraints of Assumption B.1 but do not jointly reproduce the box’s extreme edges, which is why the accepted load  $0.41$  at  $h = 0.216$  sits below the model’s  $\beta(0.216) = 0.72$ . The construction approaches every edge of the box and reaches none of them: the relaxation gap of Equation (41) measured rather than assumed.

**The busy-time ceiling, and the maximiser.** Run B multiplexes cheap declared work and roughly two covert operations per declared operation into one window, so above a busy-time ceiling  $h_{max} \approx 0.24$  its work no longer fits  $T$ . This is the capacity constraint the model already prices, evaluated at a different throughput: the clip Equation (20) converts the window’s idle seconds into covert operations at *published peak* rates, while the ceiling converts them at the rates these kernels achieve — and at achieved rates the schedule needs more seconds than the window holds. The headline  $\beta^* = 1.16$  sits at  $h^* = 0.42$ , nearly twice the ceiling, so at the headline utilisation the wall-clock budget binds before this construction reaches the corner; restricting the maximisation to  $h \leq h_{max}$  puts the model worst case at  $\beta \approx 0.77$ . The ceiling is a property of this construction, not a general feasibility bound: it is set by this kernel’s achieved rates and covert dose ratio, and includes a 10% protocol margin so the idle pad never underflows (without it,  $h_{max} = 0.27$  and the restricted worst case is  $0.82$  rather than  $0.77$ ). A different covert workload would move it, which is why the body does not quote  $0.77$  as a bound. The general point survives the caveat: a verifier reading *time* as well as energy constrains more than the energy channel alone prices, and joint energy–time inversion is future work.

**The resulting bracket.** Putting the two gaps together, the identified-set width a bare meter admits against a format-checked declaration is *at least*  $0.41$  machines — measured, attained, and accepted by the sensor — and *at most* about  $1.16$  as a model bound whose maximiser is infeasible, or about  $0.77$  if the bound is confined to a feasible window. Four-tenths of a

machine of covert compute passing an honest declaration is a wide identified set; whether it crosses the whole-machine “vacuity” line is not something the attained witness settles, and we do not claim it does. The width is set jointly by the energy budget and the silicon, not by either alone.

## F. Benchmark protocol, hardware, and provenance

Experiment 1 (Section 4.2) reads the *total energy quotient*  $E/C$  off a single A100 by holding the workload fixed and moving one factor of Equation (23) at a time. The quotient is the band-edge estimand of Section 4.2—not the marginal rate  $r = \partial P / \partial F$  of Equation (2), which Experiments 3 and 4 estimate as regression slopes—and we keep the two apart throughout. Both declared band edges are read at the same saturated rate, so their amortised baselines cancel in the band-edge difference the floor uses; the  $\approx 5\%$  spread in achieved rate across cells leaves a small residual, which widens the slack conservatively. This appendix gives the hardware, provenance, and mechanics: the workload and why its operation count is exact, how each driver is set, and how the energy is metered. Experiments 2 and 3 reuse the same protocol with a different swept axis; Experiment 3 reuses its metering inside fixed-length composite windows (declared burst + covert dose + idle pad).

**Hardware and provenance.** Experiments 1–3 and 5 ran sequentially on one NVIDIA A100-SXM4-80GB (driver 595.84), in a single Slurm allocation on the OzSTAR facility on 22 July 2026 (340, 85, 70, and 135 recorded windows respectively). The released records and capture scripts index experiments by capture identifier, which does *not* match the numbering used here: the captures are named in the order they were written, the experiments in the order the argument needs them. The mapping is Experiment 1  $\rightarrow$  exp1\_exchange, Experiment 2  $\rightarrow$  exp3\_overhead, Experiment 3  $\rightarrow$  exp7\_marginal, Experiment 4  $\rightarrow$  qblock\_marginal, and Experiment 5  $\rightarrow$  exp2\_operand. Two identifiers carry no experiment number at all: exp4\_core is the cross-device sweep and qblock the trained-weight total quotient reported only as a contrast to Experiment 4’s marginal edge. Read the identifiers, not the digits in them; the repository’s README carries the same table. Every record carries the device UUID, sweep id, configuration snapshot, and git commit, and the band-extraction code refuses to mix devices silently; because Experiment 3’s marginal cost and Experiment 1’s declared band price the same floor, the guard requires them to share one device. Seven cards of one SKU appear in total, by UUID prefix: GPU-ac1cbae9 (Experiments 1–3 and 5); GPU-4bb93dfb, GPU-617f2717, GPU-aa412071 (cross-device sweep); GPU-757ff02a (the trained-weight total quotient, reported only as a contrast to the marginal edge) and GPU-747045fa (Experiment 4, the incremental useful-work edge that prices the bottom two rungs of Section 5); GPU-108c1549 (the matched-energy witnesses of Section 4.2). Each single-device band is therefore compared against declared edges measured on a different card, and the same-SKU spread (2–3% on every band edge; Section 4.2) bounds the device variation it inherits. Full manifests accompany the released records; the trained weight/activation tensors themselves are not committed, but the manifests record their SHA-256 hashes and the GPU pre-flight validation metrics, so the extraction is checkable against the release but not replayable from the repository alone.

**Operating point.** The platform denied clock locking and power capping, so DVFS is a recorded axis rather than a controlled one: each window logs the governor-selected SM clock (achieved 1335–1410 MHz on the band sweeps), the bands are stratified into 45 MHz clock strata, and no window was flagged with a *harmful* throttle (thermal or hardware power brake). The densest operand bursts do sit at the card’s software power cap by design, which truncates their measured power and is therefore conservative for a most-expensive-rate edge.

**Meter accuracy and cadence.** nvmlDeviceGetTotalEnergyConsumption exposes a cumulative millijoule register integrated in firmware. NVML publishes no accuracy specification for this counter on GA100; the documented  $\pm 5\%$  applies to the power readings of the earlier Fermi and Kepler generations (NVIDIA Corporation, 2026), and we adopt that figure as a working meter tolerance wherever one is needed, always in the direction that widens the bound. The counter’s underlying power telemetry updates on a  $\sim 100$  ms cadence and samples only part of the runtime (Yang et al., 2023); each measured cell therefore fills a  $\sim 1.5$  s window ( $\gg 100$  ms), as described under *Metering the energy* below.

**The workload and its exact operation count.** Every cell runs a dense square matrix multiply  $AB$  with  $A \in \mathbb{R}^{M \times K}$ ,  $B \in \mathbb{R}^{K \times N}$  (we take  $M = N = K = n$ ). Each of the  $MN$  output entries is a dot product of length  $K$ — $K$  multiplies and  $K-1$  adds—so by the standard two-operations-per-multiply-add convention the matmul costs

$$C = 2MNK = 2n^3 \text{ op}, \quad (43)$$

counted as nominal operations of whatever format the cell runs (int8 multiply-adds included; Section 2.1), known *analytically* from the shapes alone, with no profiler or hardware counter involved. A measured window runs `iters` such matmuls back to back, so its nominal compute is  $C_{\text{win}} = 2n^3 \text{ iters}$  and the energy per operation is  $r = E_{\text{win}}/C_{\text{win}}$ . Because  $C$  is exact under the nominal convention (the literal scalar count is  $MN(2K - 1)$ , within 0.03% of  $2MNK$  at the band cells'  $K = 2048$ ),  $r$  inherits the meter's accuracy directly; and because  $r$  is intensive (energy *per* operation), cells of different size and `iters` compare directly.

**Setting the swept drivers.** The floor reads only the extremes of  $r$ , so what the sweep needs from each factor of Equation (23) is its *range*, not a typical value: every axis below is built to drive its factor to both extremes. Each must also move without disturbing the operation count Equation (43), or the quotient  $r = E/C$  would change for two reasons at once—a constraint that dictates the design of the locality and operand axes in particular.

- **Precision**  $\kappa$  is the tensor dtype: `fp32`, `tf32` (fp32 operands routed through the tf32 tensor-core path), `fp16`, `bf16`, and `int8` (an integer-matmul kernel,  $\text{int8} \times \text{int8} \rightarrow \text{int32}$ ). Lower precision charges less capacitance per multiply-accumulate, lowering  $\kappa$ .
- **Locality**  $C_{\text{eff}}$  is operand *residency* at a byte-identical shape, not problem size: sweeping it by changing  $n$  would change the operation count and confound the comparison, so instead we fix the shape at  $n = 2048$  and vary how many distinct operand sets the window rotates through iteration to iteration. The A100 has a 40 MB L2; at  $n = 2048$  an fp16  $A$ - $B$  operand set is  $\sim 16$  MB, so:
  - *resident* (one operand set, reused every iteration): the working set fits the L2, so most traffic is served on-chip and HBM activation is avoided: low  $C_{\text{eff}}$ ;
  - *streamed* (eight operand sets, round-robin): a reuse is evicted before it recurs, so every iteration streams fresh tiles from HBM, engaging its arrays, peripheral logic, and I/O: high  $C_{\text{eff}}$ .

Both arms run the identical shape, kernel, and operation count Equation (43); only the memory-traffic path differs, so locality no longer confounds the count. The kernel stays compute-bound in both arms—a  $2048^3$  matmul is high arithmetic intensity—so streaming raises  $C_{\text{eff}}$  without making the workload memory-bound, and Figure 5 accordingly shows locality moves  $r$  only slightly. The residency margin is comfortable only for the two-byte formats. An fp32 or tf32 operand set stores four bytes per element, so it is twice the size,  $\sim 34$  MB against the 40 MB L2: its resident arm marginally spills, and the locality contrast we measure for those two formats is attenuated accordingly.

The third driver, supply voltage  $V^2$  (DVFS), we could not sweep: the platform denied user clock-locking, so the clock was left to the governor and merely recorded per run. The operand *values* are a fourth axis: Experiment 1 crosses all six operand distributions (Appendix G) with precision and locality in a full factorial, so the pooled envelope already contains the operand spread; Experiment 5 re-runs the operand axis alone, at fixed precision and locality, as the descriptive operand-only map of Appendix G.

**Metering the energy.** Energy is read from the on-die NVML total-energy counter: the window energy is one subtraction  $E_1 - E_0$ , with a `cuda.synchronize()` fence bracketing each latch so the delta spans exactly the window's kernels; where the counter is unavailable we integrate the power query in a background sampler. Each cell fills a  $\sim 1.5$  s window with matmuls run back to back on pre-allocated buffers, with no host traffic inside the window: long enough that counter quantisation is negligible, and saturated enough that the joules are the datapath doing arithmetic, not allocation or transfer. A few warm-up windows absorb JIT and clock ramp, several measured windows follow, and the per-cell median is kept; shuffling the cell order and periodically re-measuring a fixed reference cell guards against slow thermal drift. The largest per-cell median over the sweep is  $r_{\text{hi}}^{\text{dec}}$  with no format pinned, and the largest within one format is that format's own ceiling; Figure 5 plots the full set.

## G. How Experiment 5 varies the operands

Experiment 5 holds the kernel, shape, and precision fixed and moves *only the operand values*. Every configuration runs the identical matmul  $A B$  with the identical operation count Equation (43), so the energy differences track the switching-activity factor  $\alpha$  of Equation (23)—how many bits physically toggle in the datapath as the multiply-accumulates stream through. A datapath that sees the same bits over and over charges little capacitance; one whose bits flip on every operation charges the most. The six distributions are constructed to walk  $\alpha$  from its floor to its ceiling.

**Why this experiment is reported here and not in the body.** It measures a width the body already has. Experiment 1’s factorial varies the same six operand families *inside every format*, so its fp16 cells contain this experiment’s comparison as one slice of a larger grid—and contain it at a matched clock, which this sweep does not have. Experiment 5 is therefore dominated: fp16-only, streamed-only, and confounded where Experiment 1 is balanced. We keep it for two reasons, neither of them load-bearing. It corroborates  $w_\alpha$  from independent data, and it resolves the activity factor into named distributions, which is what the threat-model rungs of Section 5 act on when they exclude zero operands. No security bound rests on any number below.

**The measurement.** The same matmul at fixed kernel, shape, and precision (fp16), varying only the operand values. Operand values alone spread  $r$  by an observed factor  $w_0 = 1.83$  (Figure 9), following the ordering zeros < low-entropy < declared-typical  $\approx$  dense-random  $\approx$  structured-sparse < adversarial-toggle. The clock confounds the comparison: locking is privilege-denied (Section 4.2) and the families did not run at a common clock—the cheapest (zeros, low-entropy) drew the highest clocks and the most expensive the lowest, a monotone 1410  $\rightarrow$  1324 MHz slide against operand cost. Experiment 1’s operand families, by contrast, share a median 1410 MHz.

That confound predicts its own direction, which is what makes the corroboration worth something. The cheap families ran fastest, so a common clock would push their energies up and spread the distribution *further*:  $w_0$  should sit *below* a matched-clock measurement of the same factor. It does—1.83 against Experiment 1’s  $w_\alpha = 1.88$ , agreeing to 3% with the residual on the predicted side. Restricting Experiment 1 to the fp16 streamed cells that match this sweep’s configuration exactly gives 1.94, bracketing  $w_0$  the same way. We offer the direction argument as a bias note rather than a proof: DVFS voltage scaling, throughput, and baseline-power amortisation pull in competing directions, and one-sided uncertainty cannot repair an uncontrolled confound.

The minimum-energy run (zeros) puts the fp16 operand floor at 1.16 pJ/op as a total quotient—a descriptive edge only; the covert denominator the floor actually uses is Experiment 3’s marginal cost (a one-sided 95% lower confidence limit), which strips the baseline power this quotient amortises in.

**The distributions.** Below we show each on a  $4 \times 4$  patch; the real operands are the same patterns at full size, generated by `operand_fill` in `powertoflops/bench/workload.py`.

**1. Zeros—the activity floor.** Every entry is zero, so no bits ever change and  $\alpha$  is at its minimum:

$$A_{\text{zeros}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

This is the cheapest run at fixed fp16 and sets the descriptive fp16 operand floor, 1.16 pJ/op as a total quotient; combined with the int8 format in Experiment 1’s factorial it reaches the cheapest measured quotient, 0.786 pJ/op, which is  $r_{10}^{\text{dec}}$  with no format pinned. (A declared entry. The covert entry  $r_{10}^{\text{cov}}$  is Experiment 3’s marginal cost, not either of these quotients; Section 4.2 and Table 1.)

**2. Low-entropy—a few distinct levels.** Entries are drawn from only four values  $\{0, 1, 2, 3\}$ . With so few distinct bit patterns, few bits differ between neighbouring operands, so operand-bit entropy—and hence switching—stays low:

$$A_{\text{low-entropy}} = \begin{pmatrix} 1 & 0 & 3 & 1 \\ 2 & 2 & 0 & 3 \\ 0 & 1 & 1 & 2 \\ 3 & 0 & 2 & 1 \end{pmatrix}.$$

**3. Structured-sparse—half the operands forced to zero.** Dense random values, but a fixed half of the columns are deterministically zeroed (every other column), giving exactly 50% zeros in a regular pattern: the structured sparsity real accelerators exploit, rather than random dropout. The persistent zero columns roughly halve the switching relative to a fully

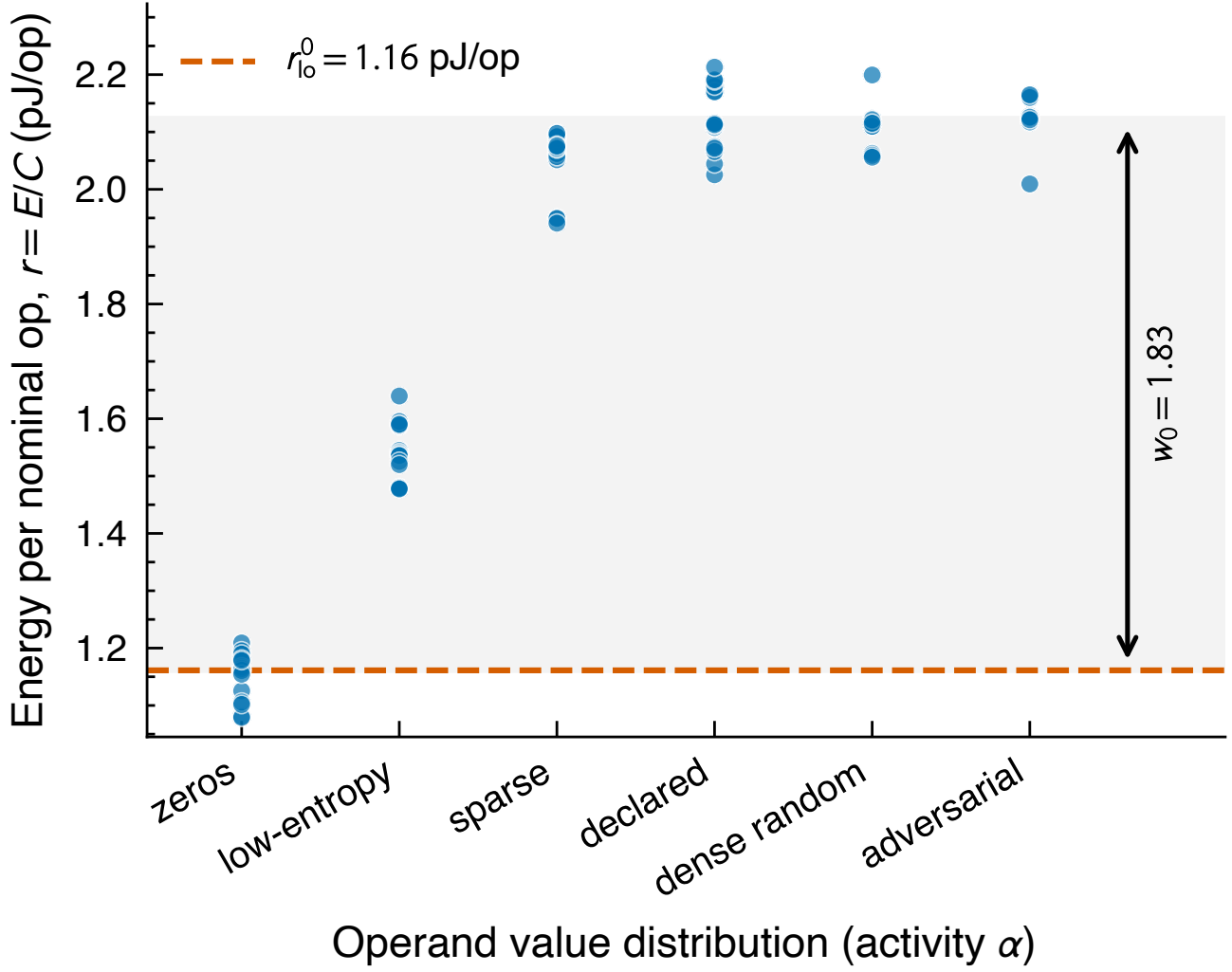


Figure 9. The shape of the activity factor  $\alpha$  (Experiment 5,  $n = 96$  windows across six operand distributions). Kernel, shape, and precision (fp16) are held fixed and only the operand values vary; the clock was governor-selected, not locked, and slides monotonically against operand cost, so this map is descriptive. The shaded span is the observed spread  $w_0 = 1.83$ , just below Experiment 1’s matched-clock  $w_\alpha = 1.88$ , on the side the confound predicts. The minimum (zeros, dashed) is the descriptive fp16 operand floor 1.16 pJ/op as a total quotient—not the covert denominator, which is Experiment 3’s marginal edge.

dense operand:

$$A_{\text{sparse}} = \begin{pmatrix} 0 & 0.4 & 0 & -0.9 \\ 0 & -0.7 & 0 & 0.2 \\ 0 & 0.8 & 0 & 0.5 \\ 0 & -0.3 & 0 & -0.6 \end{pmatrix}.$$

**4. Dense-random—generic busy data.** Every entry is an independent uniform draw from  $[-1, 1]$ : no zeros, full-width mantissas, high but not maximal switching. This is the realistic-workload reference:

$$A_{\text{dense}} = \begin{pmatrix} 0.31 & -0.82 & 0.55 & -0.14 \\ -0.67 & 0.09 & -0.48 & 0.73 \\ 0.21 & 0.64 & -0.95 & 0.38 \\ -0.52 & -0.27 & 0.86 & -0.41 \end{pmatrix}.$$

**5. Declared-typical—the realistic-activation stand-in.** Every entry is an independent standard-normal draw,  $\mathcal{N}(0, 1)$ : a proxy for the activations and weights a genuine declared workload streams, with full-width mantissas and no engineered

structure. Its switching sits alongside dense-random—high but not maximal—and it is the operand class that characterises the honest-window energy scatter used by the declared-work model:

$$A_{\text{decl}} = \begin{pmatrix} 0.42 & -1.13 & 0.27 & -0.68 \\ -0.35 & 0.91 & -1.42 & 0.11 \\ 1.06 & -0.24 & 0.58 & -0.83 \\ -0.72 & 0.39 & -0.16 & 1.24 \end{pmatrix}.$$

**6. Adversarial-toggle—the activity ceiling.** Engineered for maximal switching: the matrix is a checkerboard of two *complementary* bit patterns,  $0x5555\dots$  and its bitwise inverse  $0xA555\dots$ , so adjacent operands differ in *every* bit (maximal Hamming distance) and the operand path sees maximal input-level switching per operation. For floats the pattern is built in an integer bit-view of the same width and reinterpreted as the float type, so the raw bits—not the numeric value—are controlled exactly. Writing the two patterns as  $p$  and  $\bar{p}$ :

$$A_{\text{adv}} = \begin{pmatrix} p & \bar{p} & p & \bar{p} \\ \bar{p} & p & \bar{p} & p \\ p & \bar{p} & p & \bar{p} \\ \bar{p} & p & \bar{p} & p \end{pmatrix}, \quad p = 0x5555\dots, \quad \bar{p} = 0xA555\dots$$

A matmul forms each output as a dot product  $\text{out}[i, j] = \sum_k A[i, k] B[k, j]$ , streaming  $k = 0, 1, 2, \dots$  and loading one operand into the multiplier’s input register each cycle. Because every row and every column of the checkerboard alternates  $p, \bar{p}, p, \dots$ , that register walks

$$\underbrace{01010101}_{k=0} \rightarrow \underbrace{10101010}_{k=1} \rightarrow \underbrace{01010101}_{k=2} \rightarrow \dots$$

flipping *all* its bits on *every* cycle. Dynamic CMOS energy is paid per bit-transition, so this maximises the switching the operand registers can drive: a  $w$ -bit operand toggles  $w$  bits per cycle, against  $\approx w/2$  for dense-random (two random words differ in about half their bits) and 0 for zeros or any constant operand (the register never changes). The checkerboard is simply the 2-D layout that makes the row sequence (feeding  $A$ ) and the column sequence (feeding  $B$ ) both alternate this way at once, for every output entry.

The energy-per-operation spread from the cheapest patch ( $A_{\text{zeros}}$ ) to the densest operands (dense-random, declared-typical, and the adversarial checkerboard), at fixed operation count, is the observed operand spread  $w_0 = 1.83$  of Figure 9. The measured ordering follows the textbook one, zeros < low-entropy < structured-sparse < dense-random  $\approx$  declared-typical  $\approx$  adversarial-toggle. Per the clock confound of Section 4.2, the spread is descriptive within-device evidence, not a certified population edge—and maximal input-level toggling does not itself certify maximal switching inside the Tensor Core internals, whose operand encodings are opaque (nor do zero operands mean no hardware bits switch); no security bound in the paper rests on it.

## H. Scratch space

The two are not symmetric, though, despite entering Equation (21) as a bare minimum. Only the energy width *creates* ambiguity: pin the declared band and the overhead ( $r_{\text{hi}}^{\text{dec}} = r_{\text{lo}}^{\text{dec}}$ ,  $\Delta E_0 = 0$ ) and  $\beta_{\text{en}}$  vanishes while the clip still stands at  $(1 - h)\gamma$ , yet  $\beta = 0$ —free capacity hides nothing once the meter converts joules straight back into operations. The clip never adds width; it only caps the width that blindness has already opened. That asymmetry is why we quote  $\beta_{\text{en}}$  on its own as a diagnostic— $\beta_{\text{en}} > 1$  says the energy budget alone cannot rule out a machine’s worth of covert work—but only  $\beta$  is the width of the identified set.

**The worst-case setting.** The weakest verifier we consider is a bare power meter, which reports total energy and nothing more: to it an operation is an operation, declared or covert. It therefore takes the widest setting of the triple Equation (14): the declared edges ( $r_{\text{hi}}^{\text{dec}}, r_{\text{lo}}^{\text{dec}}$ ) open to the most expensive and cheapest per-operation costs the silicon reaches at all, and  $r_{\text{lo}}^{\text{cov}}$  stays exactly where it is, because no channel ever reached it. The clip is also laxer than the  $(1 - h)\gamma$  of Equation (20), because a bare meter pins no format on the *declared* side either: the cheating schedule may run the declared operations themselves in the cheap covert format, where they occupy only  $h/\gamma$  of the machine’s resource-time rather than  $h$ . Redoing the count of Section 3.2 with that smaller declared share leaves  $(1 - h/\gamma)T$  free, worth  $\gamma(1 - h/\gamma) = \gamma - h$  machines of covert work. With  $\beta_{\text{en}}$  evaluated at the widest setting of the triple, the power-only floor is therefore

$$\beta(h) = \min[\beta_{\text{en}}(h), \gamma - h], \quad (44)$$

and this rung is the deliberate exception to the single-scenario discipline of Appendix B: both declared edges open to the cross-format envelope while  $C_{\max}$  and  $h$  stay the declared-format yardstick.<sup>6</sup> We flag the rung as a cross-format envelope by intent wherever it is quoted; the two clips coincide at  $\gamma = 1$ .

**A worked example.** To put a number on the power-only floor, take  $C_{\max} = 100$  operation-units, declared work  $C_{\text{dec}} = 50$ , a rate known only to lie in  $[1.5, 2.5]$ , and an overhead band  $E_0 \in [10, 20]$ . In the triple Equation (14) that is  $(r_{\text{hi}}^{\text{dec}}, r_{\text{lo}}^{\text{dec}}, r_{\text{lo}}^{\text{cov}}) = (2.5, 1.5, 1.5)$ : the example stipulates the two floors equal purely to keep the arithmetic readable, and on real hardware the third entry sits below the second, which raises both terms (Section 4.2). A cheater runs  $C = 90$  at the efficient end  $\bar{r} = 1.5$  with the overhead idled to  $E_0 = 10$ , drawing  $E = 1.5 \times 90 + 10 = 145$ : exactly the energy of honest declared work at the expensive end under high overhead,  $2.5 \times 50 + 20 = 145$ . No alarm. The verifier cannot separate the honest 50 from the true 90—not for want of statistical power but structurally: the two hypotheses are energy-degenerate, and no test on the total can separate them. Of the 40 covert units,  $\approx 33$  hid under the rate ambiguity of the declared work (the first term of Equation (18)) and  $\approx 7$  under the overhead band (the second). In the language of Equation (7):  $C_{\text{lo}} = 50$ ,  $C_{\text{hi}} = 90$ , so  $\beta = 40/100 = 0.4$ . Equation (18) assembles the same number term by term:  $\beta_{\text{en}}(0.5) = 0.5(2.5 - 1.5)/1.5 + 10/(1.5 \times 100) = 1/3 + 1/15 = 0.4$ . The cheat is also capacity-feasible—the 40 covert units fit inside the  $(\gamma - h) C_{\max} = 50$  units the declared work leaves free (taking  $\gamma = 1$ )—so here the clip of Equation (44) is slack and  $\beta(0.5) = 0.4$ . Had the slack exceeded 50 units, the identified set would have ended at the capacity edge instead: at full declared utilisation  $\beta$  vanishes no matter how wide the rate band is.

**Back of the envelope (superseded by Section 4.2).** The order-of-magnitude estimate this paper carried before the A100 sweep existed. It is kept only as a record that the measured widths landed where device physics said they would; the two footnotes it used to carry have moved into the body, where the numbers they source now live.

Taking the factors in order: the precision scale  $\kappa$  spans roughly an order of magnitude from fp32 down to int8 (Horowitz, 2014) (our own sweep in Section 4.2 finds  $13.7\times$  from fp32 to int8 at dense operands); the operand activity  $\alpha$  adds a data-dependent factor of order two (no textbook number exists; we measure it directly in Section 4.2); locality and kernel set  $C_{\text{eff}}$ , for which we conservatively count no extra factor; and DVFS moves  $V^2$  by a factor of about 1.5. Multiplied, the most expensive per-operation cost the silicon reaches sits conservatively in the tens times the cheapest—but that span crosses numeric formats, and no single declared workload can be honestly explained at one format’s most expensive cells while its capacity is priced at another’s (Section 3.2). So the widest declared band, the bare-meter setting, is a factor of tens; pin the format and the precision factor drops out, leaving operands and DVFS, for a declared band of a factor 2–3. That width is a numerator, and the slope  $s$  divides it by  $r_{\text{lo}}^{\text{cov}}$ , which the same physics puts a further factor below the declared floor; so  $s$  is of order one and the ambiguity term reaches, not dwarfs, one machine. The overhead term, which we cannot fix from first principles (Section 4.1), is additive and only raises the slack. On paper, then, energy binds at low utilisation, the capacity clip of Equation (20) takes over at high utilisation, and the worst-case floor sits at a substantial fraction of the machine—up to about its declared-format capacity as a model bound. A bare power meter certifies little about how many operations ran. We put numbers on this next by direct measurement.

Note that  $\beta$  is a floor, not a measurement error. It is the ambiguity that survives a perfect meter: because distinct workloads draw identical power, no reading of the trace can tell them apart because the data hold no information that separates them. Averaging, longer observation, and better estimators all fight measurement noise, and none of them lowers  $\beta$ .

The two edges therefore also have to be *measured* differently. The declared edges enter only through their difference, so a baseline amortised equally into both cancels and a total quotient  $E/C$  serves; in the covert floor nothing cancels, so it must be measured as the *marginal* cost of an added covert operation. Appendix C states the condition precisely, and it is weaker than the affine map of Equation (2): the floor needs only that each added covert operation draws at least  $r_{\text{lo}}^{\text{cov}}$ , whatever the true power law.

<sup>6</sup>The combination still passes the single-schedule test, since the honest acceptance ceiling is the fp32 rate and fp32 can sustain the declared rate it implies: at the bare rung’s worst case,  $h^* C_{\max} = 14.3$  Top/s against a measured 17.7 Top/s fp32 peak (Section 4.2).